Università degli Studi di Torino
Dipartimento di Informatica



Tesi di Laurea Magistrale in Informatica

A Probe-Based Framework for Dynamic Real-Time Global Illumination

Relatore:
Prof. Cavagnino Davide

Candidato:
Novello Gabriele

Controrelatore:
Prof. Lucenteforte Maurizio

Anno Accademico 2025/2026

Dichiaro di essere responsabile del contenuto dell'elaborato che presento al fine del conseguimento del titolo, di non avere plagiato in tutto o in parte il lavoro prodotto da altri e di aver citato le fonti originali in modo congruente alle normative vigenti in materia di plagio e di diritto d'autore. Sono inoltre consapevole che nel caso la mia dichiarazione risultasse mendace, potrei incorrere nelle sanzioni previste dalla legge e la mia ammissione alla prova finale potrebbe essere negata.

I declare to be responsible for the content I'm presenting in order to obtain the final degree, not to have plagiarized in all or part of, the work produced by others and having cited original sources in consistent way with current plagiarism regulations and copyright. I am also aware that in case of false declaration, I could incur in law penalties and my admission to final exam could be denied.

Abstract

Accurately simulating light transport to compute global illumination remains a major challenge in computer graphics. While path-tracing offers an elegant, physically-based formulation for solving the rendering equation, its high computational variance and slow convergence still make this solution unfeasible in a real-time context.

The recent addition of dedicated ray tracing cores on modern graphics hardware enabled the proliferation of hybrid solutions combining ray tracing with already established rasterization techniques. Motivated by these technological advancements and drawing inspiration from recent academic literature and industry practices, we present a hybrid, theoretical framework to compute real-time global illumination for both dynamic lights and objects.

This method relies on a cascaded probe volume data structure, serving as a radiance and irradiance cache within the virtual scene. Alongside establishing the underlying theoretical background, this thesis focuses on detailing the inner workings of this data structure, how each probe is updated, and how these updates are scheduled and distributed across the volume's light probes under a fixed ray budget. We also briefly discuss visibility and occlusion strategies to prevent light bleeding artifacts. Finally, we formalize how these caches are queried, reconstructed, and interpolated to compute multi-bounce indirect diffuse and glossy specular reflections during the final shading stage of a standard deferred rendering pipeline.

"We observed that all nature manifests itself by means of colours to the sense of sight. We now assert, extraordinary as it may in some degree appear, that the eye sees no form, inasmuch as light, shade, and colour together constitute that which to our vision distinguishes object from object, and the parts of an object from each other."

— Johann Wolfgang von Goethe
Theory of Colours (translated by C. L. Eastlake, 1840)

Dedicated to my parents and friends,
and to the memory of Italo, Gianna and Mino.
May the light always guide thy way.

Contents

1 Introduction **10**

2 Background **18**
2.1 Monte Carlo Methods 18
2.2 Image-based Lighting 19
2.2.1 Precomputing Specular Radiance 21
2.2.2 Precomputing Diffuse Irradiance 23

3 The Probe Volume **26**
3.1 Overview 26
3.2 Structure and Representation 27
3.3 Placement Strategies 31
3.4 Sampling and Interpolation 32

4 Updating Probes **35**
4.1 Generating Rays 36
4.2 Radiance Sampling 38
4.2.1 Irradiance Cache Derivation 39
4.2.2 Prefiltered Radiance Cache Derivation 40
4.3 Update Policies 42
4.4 Visibility and Occlusion Handling 44
4.4.1 Visibility and Backface Filtering 44
4.4.2 Occlusion Handling and Surface Biasing 46

5 Surface Shading **49**
5.1 Indirect Diffuse Illumination 49
5.2 Indirect Specular Illumination 54

6 Conclusion **57**

A Implementation Details **59**
A.1 Overview 59
A.2 Architecture and Design 60
A.3 Framework Integration 63

List of Acronyms **65**

Bibliography **66**

Chapter 1

Introduction

Human visual perception would not exist without the presence of light. Light allows us to see the shape of the trees outside the window, the blue hue of the sky above our heads, and even the text written on this very page, whether printed on paper or rendered through a digital screen. Our eyes play a major role within this context, acting as an interface, a lens between ourselves and the surrounding physical world. They function as receptors, capturing light bouncing off nearby surfaces and translating it into a neural signal that is then transmitted and interpreted by the brain. In this sense, vision is not merely a passive recording of our surroundings, but an active reconstruction that shapes our entire mental model of the physical world.

In the physical sense, light is defined as electromagnetic radiation to which the human retina reacts. While physics often uses the term more broadly to refer to electromagnetic radiation of any wavelength, including infrared, ultraviolet and X-rays as well, throughout this work we use the term *light* for brevity to denote only the portion of the electromagnetic spectrum captured by our eyes, that is, visible light[1].

If an object, given a sufficient amount of energy, has the ability to produce light, it is known as an *emitter*. Two important examples of emitters throughout human history have been both fire and, of course, the Sun. Emitters act as light sources, which in turn enable our perception of the environment. Without emitters there is no light and with no light there is no vision.

Emitters produce light in discrete packets known as photons, which then continuously interact with matter through two main phenomena, scattering and absorption, until finally reaching our eyes. Absorption converts photon energy into heat, influencing both the surface perceived color and darkness, whereas scattering instead alters the propagation direction of light. Indeed, surface reflections, refractions, translucency, and subsurface scattering are all specific manifestations of scattering occurring at, or beneath a surface boundary. Everything is scattering [2].

[1] The human eye captures only a portion of the entire light spectrum, encompassing wavelengths from about 380 to 750 nanometers. These boundaries may vary on an individual basis.

In modern physics, photons are accepted to behave both as waves and as particles, and they are most accurately modeled through the laws of quantum mechanics. To our interest though, we disregard the wave behavior of light and focus solely on its particle interpretation.

One of the primary matters of computer graphics is thus that of simulating the behavior and interaction of light particles on a digital machine, with the goal of producing synthetic images. Indeed, we live in a continuous world that we cannot completely simulate on a discrete machine working with discrete quantities. To bridge this gap, we need a method, or more specifically in this context, an algorithm that, given a virtual scene, helps us compute the total amount of light reaching our virtual eyes (commonly referred to as a virtual camera, or just camera) to form the final image.

To better understand how such algorithms work, it is first necessary to distinguish between the two main types of illumination, that is *direct* and *indirect illumination*. Direct illumination accounts for photons traveling straight from an emitter to a surface, thus illuminating it directly. Indirect illumination, on the other hand, consists of photons bouncing off one or more intermediate objects before finally reaching the surface.

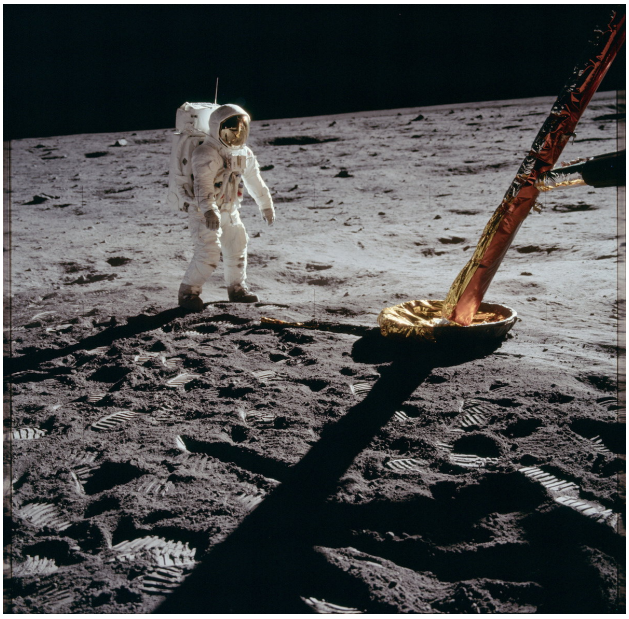
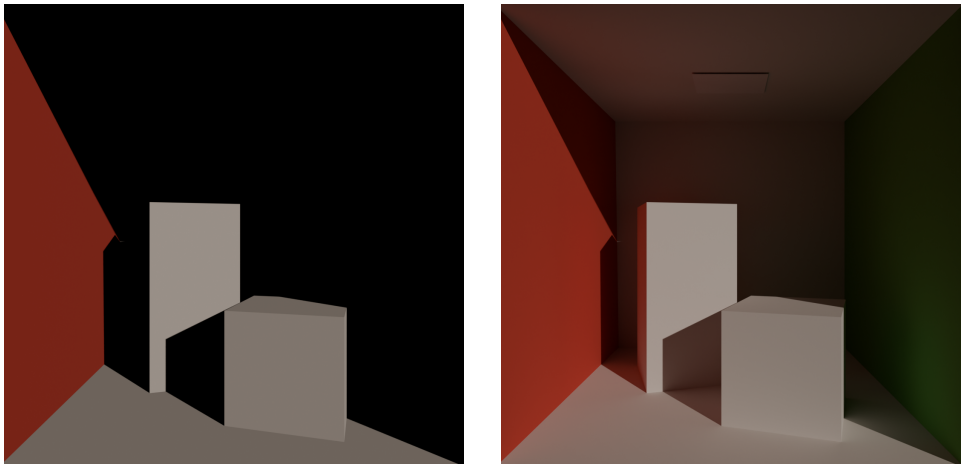


Figure 1: Photograph taken on the Moon during the Apollo 11 mission. Here, with the Sun being the sole light source and in the complete absence of atmospheric scattering, shadowed regions appear as completely black areas, lacking any surface details, which can instead be observed in the surrounding lit areas. (Source: NASA, Apollo 11 mission, 1969)

Without indirect light, shadowed areas (not directly illuminated areas) would appear completely pitch-black and lacking any details, as shown, for example, in Figure 1. Indirect illumination is also responsible for many everyday visual phenomena such as soft shadows or color bleeding (where a brightly colored object casts its hue onto neighboring surfaces). Accounting for this kind of illumination will then produce more realistic synthetic images (see Figure 2).



(a) Direct lighting only (b) Direct + indirect lighting

Figure 2: Render of the Cornell Box scene using Blender's Cycles, illuminated by a single light source simulating the Sun. The left image (a) displays direct illumination only while the right image (b) includes both the direct and indirect components, resulting in a more realistic look. Also notice how the red color from the left wall bleeds onto the white box.

To create those images, we must first formalize the underlying light transport problem. The fundamental quantity a rendering algorithm computes is the outgoing radiance[2] $L_o(\mathbf{p}, \omega_o)$ reflected from a surface point $\mathbf{p} \in \mathbb{R}^3$ towards the virtual camera along direction ω_o. Evaluating this quantity for every ray passing through the virtual camera determines the final color of each pixel of the final image, a process typically known as *rendering*. An elegant, mathematical formalization to this problem was introduced by James Kajiya in 1986 [20] through an integral equation, known as the *rendering equation*:

$$L_o(\mathbf{p}, \omega_o) = L_e(\mathbf{p}, \omega_o) + \int_{\omega_i \in \Omega} L_o(r(\mathbf{p}, \omega_i), -\omega_i) f(\mathbf{p}, \omega_i, \omega_o)(\mathbf{n} \cdot \omega_i) \, \mathrm{d}\omega_i \quad (1.1)$$

Here, the outgoing radiance is obtained by summing the light contribution from two distinct terms. The first is the emission term $L_e(\mathbf{p}, \omega_o)$, accounting for light directly emitted towards ω_o from the surface itself, while the second term is an integral over the hemisphere Ω, centered at $\mathbf{p}$ and aligned with the surface normal $\mathbf{n}$, that evaluates the portion of scattered light reflected

[2] In radiometry, radiance measures the quantity of light traveling along a single direction, defined formally as flux per unit projected area per unit solid angle ($\mathrm{W} \cdot \mathrm{m}^{-2} \cdot \mathrm{sr}^{-1}$).

towards the camera. Within this integral, $f(\mathbf{p}, \omega_i, \omega_o)$ is the bidirectional reflectance distribution function (BRDF) governing the fraction of light arriving from direction ω_i and scattered toward ω_o. The term $L_o(r(\mathbf{p}, \omega_i), -\omega_i)$ expresses the incident radiance arriving at $\mathbf{p}$ in terms of the outgoing radiance leaving another surface point $r(\mathbf{p}, \omega_i)$ along $-\omega_i$, where r is the *ray casting function* returning the first visible surface point intersected along direction ω_i [2]. Finally, $(\mathbf{n} \cdot \omega_i)$ represents Lambert's cosine term, reducing the light's intensity as the incident angle tilts away from the surface normal.

Notice that within Equation 1.1, the incoming radiance at one point must be coming from another point, and the radiance at the other point must also be coming from a third point and so on. This inherently makes $L_o(r(\mathbf{p}, \omega_i), -\omega_i)$ a recursive term. Evaluating this potentially infinite chain of reflections is what constitutes the problem of *global illumination*.

Moreover, if we relax Equation 1.1 by replacing the recursive term with an explicit incoming radiance function $L_i(\mathbf{p}, \omega_i)$ and omitting the emission term (which typically belongs to the full rendering equation), it would simplify into the non-recursive *reflectance equation* [2]:

$$L_o(\mathbf{p}, \omega_o) = \int_{\omega_i \in \Omega} L_i(\mathbf{p}, \omega_i) f(\mathbf{p}, \omega_i, \omega_o)(\mathbf{n} \cdot \omega_i) \, \mathrm{d}\omega_i \tag{1.2}$$

where $L_i(\mathbf{p}, \omega_i)$ serves as a generalized, non-recursive term denoting incoming radiance from direction ω_i. Solving this simplified form of the integral under the assumption that incoming light originates exclusively from emitters constitutes the problem of *local illumination*. In contrast with global illumination, local illumination independently computes outgoing radiance from each surface point $\mathbf{p}$ without requiring additional information from other surfaces, making the computation embarrassingly parallel. Note, however, that while local models primarily serve as a method for computing direct illumination, they may also approximate indirect illumination locally by making use of an ambient term or through more complex methods such as image-based lighting (IBL), later discussed within Chapter 2.

The most popular method to solve global illumination is by far *ray tracing*. Since only a small portion of the light rays generated by emitters ends up entering our eyes (or camera), with most of them bouncing away and dispersing into the environment, simulating light in its natural forward direction is extremely inefficient. Ray tracing solves this by reversing the physical path of light, shooting rays from the camera into the scene by passing through each pixel of the discrete grid comprising the final image. For each ray, the closest surface is found and its intersection point is then checked for whether it is in shadow by shooting a secondary ray, called shadow ray,

to each light source, finding out if any object is placed in between. Indirect lighting is also computed at the intersection point by casting additional rays along the reflection and refraction directions. These rays will again intersect a surface and repeat the whole process again, recursively. To numerically solve Kajiya's full rendering equation [20] without evaluating an infinite number of recursive light paths, stochastic ray tracing algorithms, most notably *path tracing*, numerically evaluate global illumination using Monte Carlo integration (Section 2.1). Rather than branching exponentially with each surface bounce, path tracing traces individual stochastic light paths through the scene, relying on sample averaging to converge toward a noise-free image.

Ray tracing, and consequently path tracing, are the most physically-accurate methods to simulate light transport, accounting for direct lighting, indirect lighting, soft shadows, caustics and other effects, all within one elegant solution. However, despite their conceptual simplicity, the amount of computation required to render noise-free images of acceptable quality far exceeds the capabilities of current hardware, making such technique unsuitable for real-time rendering, where each frame must be evaluated and displayed within a strict 33 (30 Hz) or even 16 milliseconds (60 Hz) window. By consequence, ray tracing was historically confined to offline (i.e. non-interactive) rendering contexts such as film productions or architectural visualization. Graphics-dedicated hardware, namely the graphics processing unit (GPU), was in fact initially designed and optimized with the rasterization rendering pipeline in mind [2]. While ray tracing algorithms can be easily parallelized and mapped on GPU threads, they nonetheless run on hardware that was not built for them and compete for computational resources with standard shading and general-purpose GPU tasks.

A major step forward occurred in 2018, with NVIDIA's introduction of dedicated hardware ray tracing cores (RT cores) within the Turing architecture of their new consumer-level GPUs [33]. These new ray tracing-dedicated cores allowed for a faster scene traversal by accelerating ray/triangle intersection directly in hardware. Although this brought real-time ray tracing closer to reality, rendering full path traced-images at native resolution while ensuring an interactive frame rate still remains very difficult, with most modern implementations heavily relying on denoising and AI-driven upscaling techniques.

However, these hardware advances allowed for an increased usage of ray tracing within traditional raster pipelines to compute lighting effects that a classic raster pipeline could not easily evaluate. Within this context, hybrid global illumination solutions also emerged, approximating path

tracing results by favoring perceptual quality over absolute physical accuracy. By combining already established raster techniques with targeted ray tracing, and by leveraging the new capabilities offered by RT cores, they make most, if not all, of the current state-of-the-art real-time global illumination solutions used within the industry. The most notable being Unreal Engine 5's Lumen [52] (Epic Games), Frostbite's global illumination based on surfels [3, 6] (GIBS, Electronic Arts) and dynamic diffuse global illumination [29] (DDGI).

Inspired by these works, and by recent industry productions [27, 47], the goal of this thesis is that of extending the probe-based solution presented by Majercik et al. within the DDGI framework into a complete global illumination solution, encompassing both indirect diffuse and specular illumination. By incorporating practical solutions established in recent productions, we aim to provide a general framework in the hopes that it may serve as a guide for future developers tasked with the implementation of a dynamic global illumination system within a real-time graphics engine. While the design and integration guidelines of this framework are detailed throughout this work, a full quantitative implementation and performance benchmarking against alternative real-time global illumination techniques are left as future work. Here, in contrast with static (or baked) global illumination[3], the term dynamic denotes a system responsive not only to dynamic lighting, but also to dynamic geometry and thus moving objects within the scene.

This framework should be thought of as a modular component whose primary role is that of providing access to indirect lighting in real-time through a dedicated spatial data structure approximating the radiance field of the scene. In other words, rather than enforcing a complete rendering pipeline, this solution has been designed to be easily integrated with existing components of a typical graphics engine such as the illumination model, shadowing algorithm, additional post-processing passes and so on.

We now provide a summary of the contents discussed within this document. In Chapter 2, we begin by providing the theoretical and mathematical foundations required by the rest of the discussion, briefly reviewing Monte Carlo methods and image-based lighting (IBL) theory. In Chapter 3, we introduce the cascaded probe volume data structure, detailing its

[3]Static (or baked) global illumination encompass a family of historically important techniques where light transport is precomputed offline (e.g., with path tracing) and stored into static textures commonly referred to as *lightmaps* which are later sampled at runtime. These methods were essential for achieving high visual fidelity with limited hardware capabilities, at the cost of indirect illumination remaining strictly unresponsive to changes in lighting and geometry.

main properties, memory layout, and sampling mechanism. In Section 4, we address the ray-traced probe update pipeline across its various stages, including a discussion to determine the most suitable update policies and methods to mitigate light and shadow leaks via wrap shading and Chebyshev occlusion bounds. In Chapter 5, we then illustrate how to integrate the probe volume data structure within a deferred rendering pipeline to reconstruct multi-bounce indirect diffuse and specular lighting while shading an arbitrary surface point. Chapter 6 provides a brief summary of our work, discussing further optimizations and highlighting future improvements. Finally, in Appendix A we outline the design of our Vulkan renderer, *Felina*, followed by a discussion on integrating the proposed global illumination framework.

Chapter 2

Background

This chapter establishes the theoretical and mathematical foundations required to present our probe-based framework for solving real-time dynamic global illumination.

Section 2.1 provides a general overview of Monte Carlo methods for numerical integration, establishing the mathematical foundation for our probe volume's radiance sampling strategy. Subsequently, Section 2.2 introduces image-based lighting theory by discussing how complex ambient lighting calculations can be precomputed once and reused repeatedly at runtime through texture lookups. This establishes the main ideas that will be later adopted to construct our dynamic diffuse and specular probe caches.

2.1 Monte Carlo Methods

Monte Carlo (MC) methods are a class of computational algorithms that rely on random sampling to approximate solutions to problems that are analytically intractable or too computationally expensive to solve deterministically [13]. In the context of computer graphics, MC methods allow the simulation of complex light transport through the approximation of the rendering equation [20].

The standard Monte Carlo estimator approximates an integral by averaging N independent samples drawn from a probability distribution function $p(x)$:

$$\hat{I}_N = \frac{1}{N} \sum_{i=1}^{N} \frac{f(x_i)}{p(x_i)} \tag{2.1}$$

where $f(x_i)$ is the integrand evaluated at sample x_i. Each individual sample contribution $\frac{f(X)}{p(X)}$ is a random variable with variance

$$\sigma^2 = \mathrm{Var}\Big[\frac{f(X)}{p(X)}\Big] = \mathbb{E}\Big[\Big(\frac{f(X)}{p(X)} - I\Big)^2\Big] \tag{2.2}$$

where the notation $X \sim p(x)$ indicates that X is a random variable distributed according to (or sampled from) the probability density function

$p(x)$, and I is the true value of the integral. This single-sample variance σ^2 measures how much one sample can fluctuate around the true integral. Since the estimator $\hat{I}_N$ is the average of N independent samples, its variance decreases proportionally to $1/N$:

$$\mathrm{Var}[\hat{I}_N] = \frac{\sigma^2}{N} \tag{2.3}$$

As a result, the standard deviation of the estimator, used to represent the typical error, decreases as $1/\sqrt{N}$, meaning that four times as many samples are required to reduce the error by half, leading to a slow convergence to the true solution. Indeed, despite its conceptual simplicity, this slow convergence becomes an important limiting factor of MC methods. To address this, numerous variance reduction techniques have been developed.

Among them, *importance sampling* plays a major role by reducing variance through a better choice of the sampling distribution. Instead of sampling uniformly, samples are drawn from a distribution that better reflects the behavior of the integrand, resulting in a significant variance reduction.

In addition to modifying the sampling distribution, convergence can be further improved by altering the sampling pattern itself. Quasi-Monte Carlo (QMC) methods replace random samples with deterministically generated, *low-discrepancy sequences* (LDS) that more uniformly cover the integration domain. Here, discrepancy informally measures how uniform a set of samples cover the integration domain, by quantifying the deviation between the empirical distribution of samples and the ideal uniform distribution. Lower discrepancy indicates a more even coverage, which in turn leads to improved integration accuracy. [13]

2.2 Image-based Lighting

Environment mapping, originally introduced by Blinn and Newell [5] in 1976, is a technique to capture a spherical function (e.g. incoming radiance) by projecting it onto one or more images, commonly referred to as *environment maps*. By sampling those textures, renderers could efficiently approximate specular reflections from distant environments, without the need for expensive ray tracing calculations.

Twenty years later, image-based lighting (IBL) [11] techniques further built upon that idea by using environment maps not just to evaluate reflections, but as extended, infinitely distant light sources to illuminate virtual objects, becoming one of the most popular solution for locally simulating complex ambient lighting, known as *environment lighting*, in real-time.

However, simply querying an environment map only works for perfectly smooth, mirror-like surfaces as in the original scenario presented by Blinn and Newell, where incoming light is sampled along a single discrete direction given by the ideal reflection vector. For physically-based materials with non-zero roughness, the reflection distribution broadens into a wider specular lobe, requiring averaging multiple samples to accurately evaluate the response. To compute the outgoing radiance at a surface point under continuous environment lighting, we must now evaluate the reflectance equation across the entire hemisphere. By assuming an infinitely distant environment map, the incident radiance depends solely on direction. Consequently, we can rewrite Equation 1.2 omitting the spatial position $\mathbf{p}$ to simplify the expression:

$$L_o(\omega_o) = \int_{\omega_i \in \Omega} L_i(\omega_i)\, f(\omega_i, \omega_o)\, (\omega_i \cdot \mathbf{n})\, \mathrm{d}\omega_i \tag{2.4}$$

This notation will be kept throughout the section.

Evaluating this integral for every surface point at runtime via Monte Carlo numerical integration would require an excessive number of samples per pixel, making it computationally prohibitive for real-time applications, even when combined with optimizations like importance sampling techniques [2]. Thus, in order to simplify the problem, we can subdivide the reflectance equation into distinct components.

Assuming that surface reflections and subsurface scattering can be modeled independently, the total BRDF f can be split into an additive combination of diffuse (f_d) and specular (f_s) terms:

$$f(\omega_i, \omega_o) = f_d(\omega_i, \omega_o) + f_s(\omega_i, \omega_o) \tag{2.5}$$

Substituting this decomposition into Equation 2.4 allows us to split the reflectance equation into two independent integrals:

$$\begin{aligned} &L_o(\omega_o) = L_d(\omega_o) + L_s(\omega_o) \\ &\text{where} \\ &L_d(\omega_o) = \int_{\omega_i \in \Omega} L_i(\omega_i)\, f_d(\omega_i, \omega_o)\, (\omega_i \cdot \mathbf{n})\, \mathrm{d}\omega_i \\ &L_s(\omega_o) = \int_{\omega_i \in \Omega} L_i(\omega_i)\, f_s(\omega_i, \omega_o)\, (\omega_i \cdot \mathbf{n})\, \mathrm{d}\omega_i \end{aligned} \tag{2.6}$$

By decoupling the BRDF, the evaluation of $L_d(\omega_o)$ and $L_s(\omega_o)$ can be handled independently. Both solutions replace computationally expensive per-

pixel Monte Carlo integration with pre-integrated filtering, traditionally executed offline though equally adaptable to on-the-fly updates, effectively converting hemispherical integrals into fast texture lookups or compact functional evaluations.

Section 2.2.1 presents the split-integral approximation used to evaluate the specular component, while Section 2.2.2 addresses the diffuse component.

2.2.1 Precomputing Specular Radiance

Under the standard microfacet framework [9], reflections are modeled by assuming the surface is made up of many microscopic ideal specular mirrors and are described mathematically by the specular BRDF f_s as:

$$f_s(\omega_i, \omega_o) = \frac{D(\mathbf{h}, \alpha)\, F(\omega_o, \mathbf{h})\, G(\omega_i, \omega_o, \mathbf{n})}{4\,(\omega_i \cdot \mathbf{n})\,(\omega_o \cdot \mathbf{n})} \tag{2.7}$$

where $\mathbf{h} = \frac{\omega_i + \omega_o}{\|\omega_i + \omega_o\|}$ represents the microfacet halfway-vector, $D(\mathbf{h}, \alpha)$ the normal distribution function (NDF) describing the microfacets orientation[4], $F(\omega_o, \mathbf{h})$ the Fresnel reflectance quantifying the fraction of reflected light and $G(\omega_i, \omega_o, \mathbf{n})$ the shadowing-masking function accounting for geometric micro-occlusions.

As surface roughness α varies, $D(\mathbf{h}, \alpha)$ directly determines the size and shape of the specular reflection lobe with smooth surfaces yielding a sharp peak around the ideal specular reflection direction and rough surfaces presenting a much larger lobe. Because continuous environment lighting must be integrated over the entire specular lobe, evaluating the specular part of Equation 2.6 naively, on a per-pixel basis during rasterization, would require hundreds to thousands of directional samples per pixel. A computational burden far exceeding real-time budgets.

To avoid per-pixel Monte Carlo integration at runtime, Karis [21] introduced what is widely known as the *split-integral approximation*[5], which factors the specular reflectance integral into two independent components:

$$L_s(\omega_o) \approx \int_{\omega_i \in \Omega} L_i(\omega_i)\, D(\mathbf{h}, \alpha)\, (\omega_i \cdot \mathbf{n})\, \mathrm{d}\omega_i \cdot \int_{\omega_i \in \Omega} f_s(\omega_i, \omega_o)\, (\omega_i \cdot \mathbf{n})\, \mathrm{d}\omega_i \tag{2.8}$$

[4] A common choice is to use the GGX (Trowbridge–Reitz) distribution [50]

[5] He originally termed this the *split-sum approximation* because his derivation operated directly on discrete Monte Carlo estimator sums, but both terms denote the same solution. We express it here in the continuous integral form following *Real-Time Rendering* [2] notation.

The first integral convolves the incoming radiance with the specular reflection lobe. Because the integral depends on three parameters, the normal $\mathbf{n}$, the view direction ω_o and roughness α, pre-evaluating and storing this integral for every possible combination of these variables would require an intractable memory footprint. Karis drastically simplifies this dimensionality by removing the view dependency, assuming that viewing occurs at normal incidence where the view direction, surface normal, and mirror reflection direction coincide ($\omega_o = \mathbf{n} = \mathbf{r}$). As further analyzed by Lagarde and de Rousiers [25], this simplification forces the specular filter lobe to remain isotropic around $\mathbf{r}$ regardless of the viewing angle. Consequently, it fails to capture the natural anisotropic stretching of specular reflections at grazing angles, an artifact that is visually noticeable on flat, smooth surfaces, but far less discernible on curved geometry.

In exchange for this acceptable visual trade-off, the view-angle dependency is now eliminated, reducing the integral's parameters to just the reflection direction $\mathbf{r}$ and roughness α. This simplification thus allows the integral to be precomputed offline and stored in what is usually referred to as a *prefiltered environment map*, typically a cubemap whose higher mipmap levels correspond to increasing roughness values, yielding progressively blurred reflections. At last, evaluating this term while shading reduces to a single cubemap lookup indexed along the reflection vector $\mathbf{r}$, using trilinear filtering to sample smoothly between discrete mipmap levels for any continuous roughness α.

The second integral evaluates the *hemispherical-directional reflectance* (also referred to as *directional albedo*) of the specular term, which measures the proportion of uniform incoming light over the hemisphere that successfully reflects into the specific outgoing direction ω_o. At this point, the integral depends on the material roughness α, the viewing angle, represented by the dot product $\mathbf{n} \cdot \omega_o$ and the Fresnel term F.

The latter is typically implemented using Schlick's approximation of the full Fresnel equations [42]:

$$F(\omega_o, \mathbf{h}) \approx F_0 + (1 - F_0)(1 - \omega_o \cdot \mathbf{h})^5 \tag{2.9}$$

with F_0 being the specular color at normal incidence.

Directly precomputing this integral would require storing results for every possible material reflectance F_0, creating an intractable search space. However, Karis [21] observed that by substituting Equation 2.9 into the specular integral, the material-dependent F_0 term can be factored out of the integrand due to linearity:

$$\int_{\omega_i \in \Omega} f_s(\omega_i, \omega_o)\,(\omega_i \cdot \mathbf{n})\,\mathrm{d}\omega_i = F_0 \cdot A(\mathbf{n} \cdot \omega_o, \alpha) + B(\mathbf{n} \cdot \omega_o, \alpha) \tag{2.10}$$

where A and B are respectively the *scale* and *bias* of F_0 and they are defined as:

$$A(\mathbf{n} \cdot \omega_o, \alpha) = \int_{\omega_i \in \Omega} \frac{D(\mathbf{h}, \alpha)\, G(\omega_i, \omega_o, \mathbf{n})\, \left[1 - (1 - \omega_o \cdot \mathbf{h})^5\right]}{4\,(\omega_o \cdot \mathbf{n})}\,\mathrm{d}\omega_i \tag{2.11}$$

$$B(\mathbf{n} \cdot \omega_o, \alpha) = \int_{\omega_i \in \Omega} \frac{D(\mathbf{h}, \alpha)\, G(\omega_i, \omega_o, \mathbf{n})\, (1 - \omega_o \cdot \mathbf{h})^5}{4\,(\omega_o \cdot \mathbf{n})}\,\mathrm{d}\omega_i \tag{2.12}$$

Because A and B just depend on the viewing angle $\mathbf{n} \cdot \omega_o$ and the roughness α, with both values being in the $[0, 1]$ range, they can now be precomputed and stored in a two-channel lookup texture [21], referred to as the *BRDF integration map*, which can be sampled at runtime to evaluate the hemispherical-directional reflectance. Alternatively, Lazarov [26] and later Iwanicki and Pesce [18] proposed analytical approximations of the two factors done through function fitting. Also, notice that factoring out F_0 decouples the integration from material-specific parameters, allowing terms A and B to be evaluated only once, either offline or during engine initialization, and shared universally across all materials.

Summarizing, the split-integral approximation simplifies the computationally prohibitive specular integral into two independent precomputable factors: the prefiltered environment map and the BRDF integration map.

In classic image-based lighting, the first is usually implemented using a mipmapped cubemap and sampled at runtime through trilinear interpolation with the reflection vector $\mathbf{r}$ and surface roughness α. Concurrently, scale A and bias B are retrieved from a universal BRDF integration map indexed by $(\mathbf{n} \cdot \omega_o, \alpha)$ or computed via closed-form polynomial fits. Finally, multiplying the sampled prefiltered radiance by the reconstructed directional albedo, $F_0 \cdot A + B$, yields a physically plausible evaluation of specular environment lighting for the price of just two texture lookups.

2.2.2 Precomputing Diffuse Irradiance

The diffuse integral of Equation 2.6 accounts for the part of incoming light that penetrates the surface interface, undergoes multiple internal scattering events, and re-emerges uniformly across the hemisphere. Under the standard

assumption of an ideal Lambertian BRDF[6], the diffuse term is isotropic and defined as a constant $f_d = \frac{\rho}{\pi}$, where ρ represents the surface albedo.

Substituting f_d into the diffuse integral from Equation 2.6 yields:

$$\int_{\omega_i \in \Omega} L_i(\omega_i) \, \frac{\rho}{\pi} \, (\omega_i \cdot \mathbf{n}) \, \mathrm{d}\omega_i = \frac{\rho}{\pi} \int_{\omega_i \in \Omega} L_i(\omega_i) \, (\omega_i \cdot \mathbf{n}) \, \mathrm{d}\omega_i \tag{2.13}$$

Because ρ and π do not depend on the incident light vector ω_i, they factor out of the integral completely. The remaining integral represents the total incoming cosine-weighted hemispherical irradiance $E(\mathbf{n})$ at the surface point:

$$E(\mathbf{n}) = \int_{\omega_i \in \Omega} L_i(\omega_i) \, (\omega_i \cdot \mathbf{n}) \, \mathrm{d}\omega_i \tag{2.14}$$

Notice that $E(\mathbf{n})$ depends solely on the surface orientation normal $\mathbf{n}$, completely removing any view dependency from ω_o. Furthermore, the cosine kernel $(\omega_i \cdot \mathbf{n})$ acts as a low-pass filter over the incident radiance field L_i.

This allows $E(\mathbf{n})$ to be precomputed across a discrete set of surface orientations $\mathbf{n}$ and stored in an *irradiance environment map* (typically a low-resolution cubemap), where each texel encodes the irradiance for a surface facing direction $\mathbf{n}$. Alternatively, the smooth nature of the irradiance environment map allows it to be efficiently represented with high accuracy by third-order spherical harmonics (SH), requiring only 9 RGB coefficients (i.e. 27 floating-point values) to evaluate $E(\mathbf{n})$ analytically without requiring any texture lookups [39].

Thus, the runtime evaluation of the diffuse component simplifies to

$$L_d(\omega_\mathbf{o}) = \frac{\rho}{\pi} \, E(\mathbf{n}) \tag{2.15}$$

Combining the precomputed diffuse and specular components as in Equation 2.6 yields the complete runtime evaluation of environment lighting. While classic IBL assumes a single environment map located at infinity

[6] While more complex diffuse BRDFs exist (e.g., the Oren–Nayar model [36] or the Disney diffuse model [7]) and are frequently used for analytical direct lights, falling back to an isotropic Lambertian model for indirect lighting remains common practice. Although split-integral approximations for non-Lambertian diffuse BRDFs can also be applied to environment lighting [25], the low-pass filtering nature of hemispherical cosine integration renders the subtle view-dependent variations of complex models visually imperceptible when used as indirect illumination. We therefore adopt the standard Lambertian formulation.

and was initially introduced to simulate more complex and realistic ambient lighting, the core principles detailed here can also be applied to modern probe-based global illumination algorithms as presented in the subsequent chapters.

Chapter 3

The Probe Volume

The foundation of this framework relies on a spatial data structure which we refer to as the *probe volume*. This chapter is in fact entirely dedicated to outlining and describing this data structure, starting with Section 3.1 where we provide a brief, high-level overview of the light field concept which serves as the theoretical and historical background of the structure. Then, continuing with Section 3.2 we detail its memory representation and cascaded structure, while Section 3.3 discusses various placement strategies. Lastly, in Section 3.4 we describe how this spatial data structure can be queried to reconstruct the light field at an arbitrary point in space within the volume bounds.

3.1 Overview

First conceptualized by the Soviet physicist Andrey Gershun in 1936 (English translation, 1939 [15]) and later formalized by Adelson and Bergen [1] as the plenoptic function, a light field (also known as radiance field) is a physical field describing the amount of light flowing through a region of three-dimensional space. Because this continuous formulation may span up to seven dimensions, evaluating it everywhere constitutes a computationally intractable task, forcing algorithms to rely on some sort of spatial discretization.

Levoy and Hanrahan first brought this approach to computer graphics in their seminal paper on light field rendering [28], which Greger et al. subsequently adapted to the context of global illumination by introducing irradiance volumes [16]. Even today, probe-based representations remain central to modern real-time global illumination solutions, often serving the role of a spatial lighting cache [27, 29].

Structurally, the probe volume consists of a finite set of programmatically placed light probes, which may be thought of as suspended spheres in 3D space describing and capturing, thus probing, the behavior of light in their surrounding environment. Note, however, that even though these probes are placed within a virtual scene, they do not affect or interact with light transport in any way.

Mathematically, the volume is represented by the five-dimensional plenoptic function $L(\mathbf{x}, \omega)$ encoding the spatio-angular distribution of radiance incoming from direction $\omega \in \mathbb{S}^2$ for each light probe at location $\mathbf{x} \in \mathbb{R}^3$.

3.2 Structure and Representation

In practice, the probe volume data structure corresponds to a three-dimensional, finite grid of light probes, discretizing the continuous light field within a virtual world. More specifically, as opposed to a standard regular grid, our framework uses a more complex variant called a *cascaded probe volume*, which will be described next.

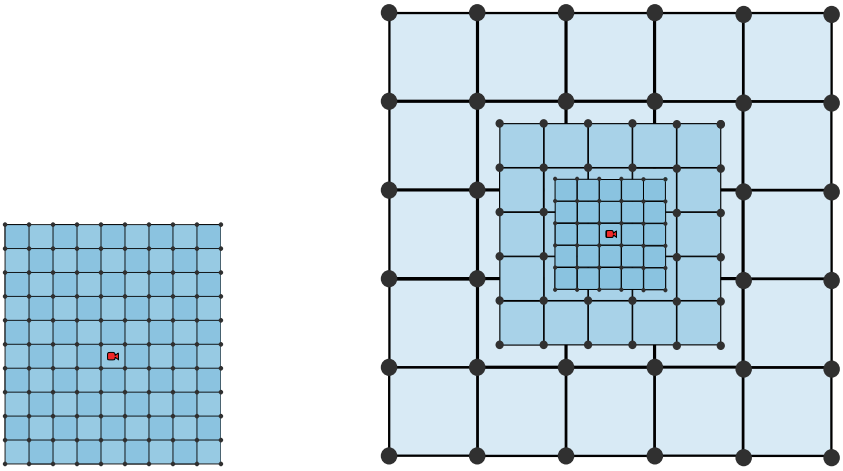
This variant is a generalization of the standard version where probes are organized into multiple and independent grid layers, referred to as cascades, where the spatial resolution of each layer is reduced with each successive level (see Figure 3). In this context, a lower resolution means an increased spacing between adjacent probes, resulting in a coarser radiance sampling, but allowing the cascade to cover a significantly larger portion of space. A common approach [27] is to double the probe spacing with each layer, such that the three-dimensional spacing of an arbitrary cascade c is defined as:

$$\mathbf{s}_c = 2^c \mathbf{s}_0 \tag{3.1}$$

where $\mathbf{s}_0 \in \mathbb{R}^3$ denotes the base spacing of the innermost cascade and $c \in \{0, \ldots, C-1\}$ specifies the target cascade level.

In practice, $\mathbf{s}_0$ is chosen based on the spatial scale of the whole scene. In line with recommendations from modern real-time global illumination systems [27, 29], for typical human-scale environments, a base spacing of one or two meters is usually a reasonable choice.

Compared to the uniform approach, where probes are evenly spaced across the entire scene, the cascaded grid concentrates light probes in regions closest to the camera, allowing for more accurate radiance sampling where detail matters, while instead reducing sampling density farther away, where high-frequency detail is less perceptually significant. This is akin to traditional geometry level of detail (LOD) techniques. Additionally, for a fixed probe budget, using cascades also provides a broader spatial coverage which may be exploited to light large open-world environment, an approach successfully leveraged in recent video game titles such as *Assassin's Creed Shadows* [27]. A visual comparison between the uniform and cascaded approach is illustrated in Figure 3.



(a) (b)

Figure 3: Comparison between a 2D top-view of a uniform and cascaded probe volume using the same probe budget (the same ideas also apply to the third dimension). The left image shows a 10×11 uniform grid for a total of 110 probes, while the right image shows a 3-level cascaded grid for a total of 108 probes (36 per cascade). The spatial resolution of the uniform grid and cascade $c = 0$ are equivalent, showing how the cascaded version allows for a larger scene coverage while using the same number of probes.

The probe volume is positioned in the virtual world as an axis-aligned grid that tracks camera movement throughout the scene, ensuring that high-density probes remain centered on the viewer. Rather than translating continuously though, an operation that would introduce temporal flickering and continuously invalidate the probe cached data, each cascade's origin independently snaps to discrete world-space increments matching its specific probe spacing $\mathbf{s}_c$, as shown in Figure 4. Consequently, probes remain stationary in world-space until the camera crosses a full grid cell. At that point, the slice of probes falling off the trailing edge is recycled to cover the newly exposed region on the leading edge, leaving all other probe locations undisturbed. This allows for possible optimizations such as caching per-probe hit surfaces data in a G-buffer-like cache[7] to remove the need, later on, for a

[7]The term G-buffer (Geometric buffer) was originally introduced by Saito and Takahashi [40] to denote a set of 2D textures used within deferred rendering pipelines [49] to store surface geometry and material properties (such as normals, depth, and albedo) per pixel during scene rasterization, thereby postponing lighting computations, which in traditional forward rendering are performed directly during rasterization, to a later pass.

full ray-traced update for probes who did not move, thus limiting it to just re-evaluate lighting (Section 4).

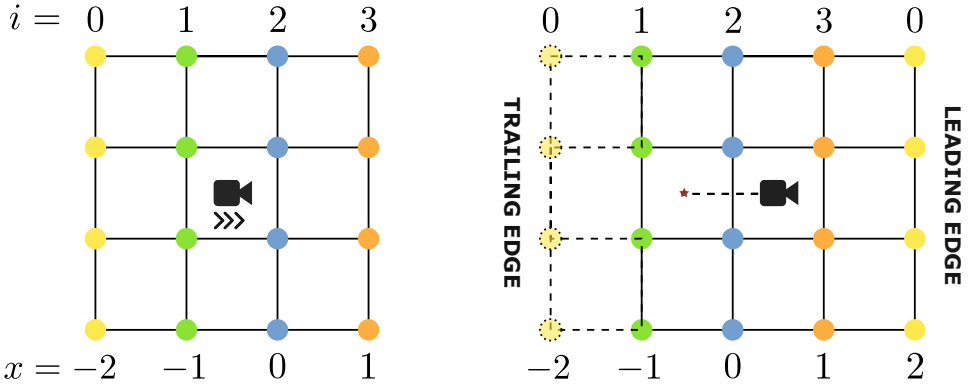


Figure 4: Top-down view of a single probe volume cascade (i.e., with constant, unit probe spacing), as each cascade is moved independently of the others. As the camera moves one cell to the right, the cascade origin snaps by its probe spacing, keeping existing probes stationary while trailing edge probes are recycled to cover the newly exposed leading edge. Under toroidal addressing (Equation 3.2), tile index $i_x = 0$ and its corresponding memory slot are recycled without disturbing the remaining green, blue, and orange probes.

To handle this incremental wrapping without costly memory shifts, each cascade employs a toroidal addressing scheme, mapping a probe's discrete world-space index $\mathbf{x} \in \mathbb{Z}^3$ to its tile coordinate $\mathbf{i} \in \mathbb{N}^3$ within the probe volume texture atlases via modulo arithmetic:

$$\mathbf{i} = \left(\left\lfloor \frac{\mathbf{x}}{\mathbf{s}_c} \right\rfloor + \frac{\mathbf{N}}{2} \right) \bmod \mathbf{N} \tag{3.2}$$

where $\mathbf{N} \in \mathbb{N}^3$ denotes the grid resolution along each axis, $\mathbf{s}_c \in \mathbb{R}^3$ represents the probe spacing for cascade c, and $\lfloor \cdot \rfloor$ specifies the component-wise floor function, mapping the continuous positions $\mathbf{x}$ from local camera-relative space to discrete grid coordinates (with vector division performed element-wise). A brief example is also provided in Figure 4.

This three-dimensional index $\mathbf{i} = (i_x, i_y, i_z)^T \in \mathbb{N}^3$ is subsequently flattened into a scalar tile index I via row-major linearization, also taking the cascade level $c \in \{0, \ldots, C-1\}$ into account:

$$I = c \cdot (N_x N_y N_z) + i_x + N_x i_y + N_x N_y i_z \tag{3.3}$$

This addressing scheme thus provides direct access to each probe's individual data tile within each of the four different texture atlases comprising the volume's complete data layer. Each of these atlases can be thought of as an array of textures, sequentially storing the probe data for cascade $c = 0$, cascade $c = 1$, and so on. A visual reference, adapted from DDGI's atlas layouts [29] is provided in Figure 5.

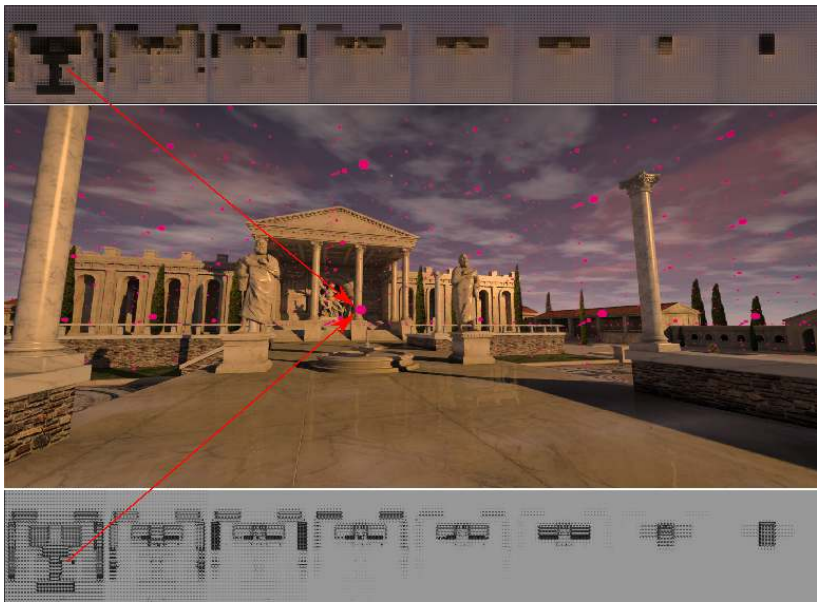


Figure 5: A visual example of the probe volume memory layout, adapted from the DDGI framework by Majercik et al. [29]. Their original layout includes two texture atlases: one storing irradiance (top) and the other storing depth (bottom). Each probe tile within the atlases is mapped using octahedral projection [8] with appropriate padding and border gutters to prevent interpolation artifacts. In our cascaded implementation, this memory layout is expanded to four distinct atlases, but the idea remains the same.

The first two of these atlases store the irradiance and prefiltered radiance texture maps. Both are derived from the raw radiance values sampled during the probe update pass through ray-tracing (Chapter 4) and are later used to compute indirect illumination (Chapter 5). The third atlas stores the first two distance moments, that is, the mean and mean squared of the ray hit distance, which are then used to estimate visibility and reduce light bleeding artifacts when sampling from the volume (Section 4.4). Finally, the last atlas stores the G-buffer caches, holding per-ray surface hit attributes, such as hit

position, surface normals, and material properties [27], which can be re-used to update the lighting caches without re-casting any rays, through a process discussed in Chapter 4.

With the exception of the discrete G-buffer cache, which stores hit attributes for a finite set of generated ray directions (e.g., an 8×8 texture would store up to 64 ray samples per probe), all the other quantities, assuming a fixed probe location, vary continuously as a function of direction $\omega \in \mathbb{S}^2$. As a result, the stored data forms a continuous spherical signal that must be encoded onto a two-dimensional texture to enable correct sampling and interpolation along arbitrary directions at runtime. While early global illumination frameworks, such as the original irradiance volume [16], used cube maps to represent spherical functions, modern real-time systems [27, 29, 35] tend to rely on octahedral maps [8]. *Octahedral mapping* is a texture mapping technique that encodes a spherical function by first projecting unit vectors onto the surface of a regular octahedron and then unfold it onto a flat, two-dimensional square domain, as illustrated by Figure 6.

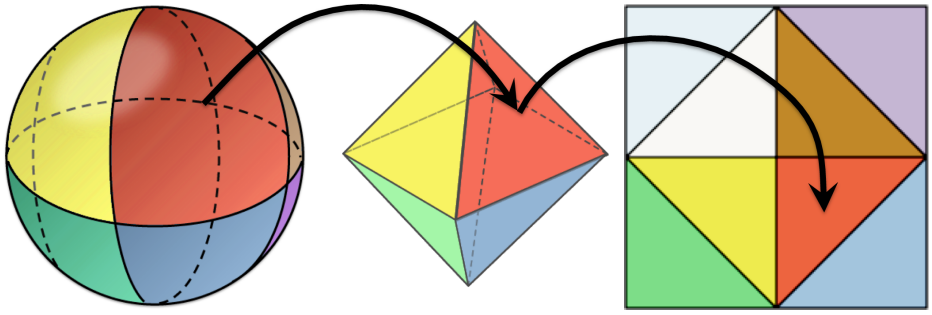


Figure 6: Intuitive overview of octahedral projection, from a spherical to a flat, two-dimensional square domain, which can be addressed using standard UV coordinates (from Cigolle et al. [8]).

Compared to traditional cube maps, this mapping provides lower spatial distortion across the sphere and significantly simplifies tile border management within a texture atlas [29].

3.3 Placement Strategies

Following the DDGI architecture proposed by Majercik et al. [29], we adopt an axis-aligned probe layout. This layout, however, is not a strict require-

ment and alternative configurations may also be taken into consideration. We briefly discuss one such alternative: tetrahedral grids [10].

Compared to a simple, regular grid, an irregular tetrahedral layout offers the possibility of using more complex and adaptive probe placing strategies, such as concentrating probes around complex geometry to avoid oversampling uniform areas. Additionally, by aligning probes with geometric surfaces it can also help reduce light and shadow leaks caused by light probes positioned inside geometry, a major drawback of regular grids. For these reasons, in small-to-medium environments, a static tetrahedral mesh spanning the entire scene remains a compelling choice. However, evaluating such an irregular structure on the GPU introduces substantial sampling and interpolation complexity, as well as a poor scaling to wider scenes, becoming memory-prohibitive very quickly.

In such large-scale contexts, a camera-centric uniform grid provides a more scalable alternative, dynamically translating with the viewer while also guaranteeing a faster addressing through trilinear interpolation. Furthermore, by structuring the uniform grid into cascades as previously described, we also partially compensate for the regular grid's lack of spatial adaptivity.

Of course, the primary drawback of this regular layout still remains its lack of geometry awareness, causing probes to inevitably fall inside solid walls, below the terrain, or behind back-faces. If these internal probes are sampled during interpolation, they introduce severe visual artifacts, such as light or shadow leaks. While some productions mitigate this by physically relocating probes outward into free space [27], by staying consistent with the DDGI architecture, we avoid physical relocation entirely. Instead, while updating the probe volume, we automatically detect probes inside geometry and assign an interpolation weight of zero, safely ignoring them during the final lighting blend operation. More details on how we mitigate leaks are provided in Section 4.4.

3.4 Sampling and Interpolation

After describing the main properties and structure of the probe volume, we now address the problem of reconstructing the continuous radiance field by sampling the data structure. This is a fundamental aspect of our framework, as it is through this sampling mechanism that we obtain indirect illumination both during the probe volume update, as detailed in Chapter 4, and shading, discussed in Chapter 5.

Sampling is always done through the use of a pair composed of a location $\mathbf{x} \in \mathbb{R}^3$ and a direction $\omega \in \mathbb{S}^2$. Whether evaluating a surface point along its normal or querying the volume at the end point of a ray's path, the location $\mathbf{x}$ must always reside within the volume bounds. Then, by definition, any point within those bounds is always contained within a $2 \times 2 \times 2$ probe cage defined by the eight nearest probes surrounding the location. In our cascaded layout, we select the eight probes belonging to the finest cascade level c whose axis-aligned bounding box (AABB) encapsulates $\mathbf{x}$, ensuring indirect lighting is evaluated using the highest spatial resolution available.

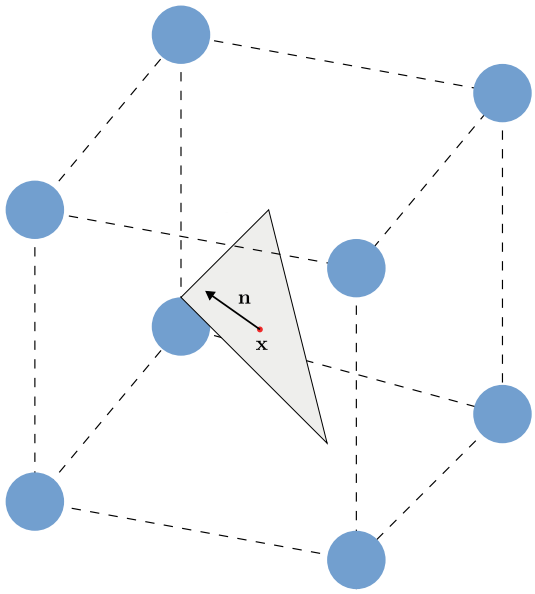


Figure 7: Illustration of a surface query. The surface point $\mathbf{x}$ is used to locate the eight nearest probes making up the surrounding probe cage. Then, the surface normal $\mathbf{n}$ is used as the sampling direction to query each probe's irradiance cache.

Once the cage is located, we iterate through each of the eight probes p_i to sample and accumulate their cached lighting data. The specific cache queried depends on the specific goal and context. In our framework, we have two main types of queries: *surface queries* and *ray queries*. A surface query uses the surface location $\mathbf{x}$ and normal $\mathbf{n}$ to evaluate indirect diffuse lighting by sampling each probe's irradiance cache $E_i(\mathbf{x}, \mathbf{n})$ (Figure 7). A ray query, on the other hand, uses the ray end point and direction ω to sample their prefiltered radiance cache $L_i(\mathbf{x}, \omega)$ to approximate incoming

directional radiance at that location.

Since irradiance and prefiltered radiance are both stored in octahedral textures (see Section 4.2), we always require converting the sampling direction, whether $\mathbf{n}$ or ω, to octahedral coordinates before sampling.

The contributions from the eight cell probes are then combined using composite interpolation weights. The total weight W_i assigned to probe p_i is computed as the product of different factors:

$$W_i = f\left(w_{\text{tri},i} \cdot w_{\text{vis},i} \cdot w_{\text{occ},i}\right) \tag{3.4}$$

where $w_{\text{tri},i}$ represents the standard trilinear spatial interpolation weight based on the shading point's position within the probe cage, $w_{\text{vis},i}$ is the directional visibility weight computed via wrap shading, and $w_{\text{occ},i}$ denotes the probabilistic occlusion weight evaluated using Chebyshev's inequality. Details to compute these last two weights are deferred to Section 4.4.

Finally, $f(\cdot)$ is a non-linear function applied to suppress low-weight contributions below a certain threshold [29, 35]. This accounts for the logarithmic response of human vision, where even minor low-intensity light leaks in dark regions are highly noticeable to the eye. In the RTXGI implementation [35], the $f(w)$ is formulated as:

$$f(w) = \begin{cases} \dfrac{w^3}{c^2}, & \text{if } w < c, \\ w, & \text{otherwise} \end{cases} \tag{3.5}$$

where $c \in (0, 1]$ denotes the crushing threshold, set to $c = 0.2$ in their implementation.

If probe p_i is classified as inactive while being updated (Chapter 4), its composite weight W_i is instead directly set to 0.

At last, the final interpolated value $F(\mathbf{x}, \omega)$, representing either the sampled irradiance or prefiltered radiance, is evaluated by normalizing the composite weights across all eight cell probes:

$$F(\mathbf{x}, \omega) = \frac{\sum_{i=1}^{8} W_i F_i}{\sum_{i=1}^{8} W_i + \epsilon} \tag{3.6}$$

where F_i is the field value sampled from probe p_i, and $\epsilon > 0$ is a small scalar placed outside the sum to prevent division by zero.

This interpolation scheme guarantees a correct and coherent method to retrieve data from the probe volume, however, its overall cost within the established frame budget must be carefully quantified. A possible optimization to further reduce its cost is later presented in Chapter 6.

Chapter 4

Updating Probes

This chapter covers different topics related to the updating process of the probe volume. The data structure should be, in fact, at least partially updated each frame to guarantee an adequate response to the dynamic components of the scene.

From an high-level perspective, the update process is subdivided in four different steps, as shown in Figure 8. The first step consists in selecting which probes require an update based on a set of criteria. If a probe is chosen, it starts executing the update pipeline by first shooting rays into the scene in order to update its G-buffer cache with new hit positions. The incoming scene radiance is then computed by evaluating lighting at each of these hit points. Once the raw, noisy radiance sample are all gathered, they are fed to an additional processing step with the goal of deriving more stable data. In other words, the raw radiance samples are thus elaborated further to yield two separate caches, the irradiance and prefiltered radiance caches, which are then stored in probe memory and later sampled to provide indirect lighting during shading.

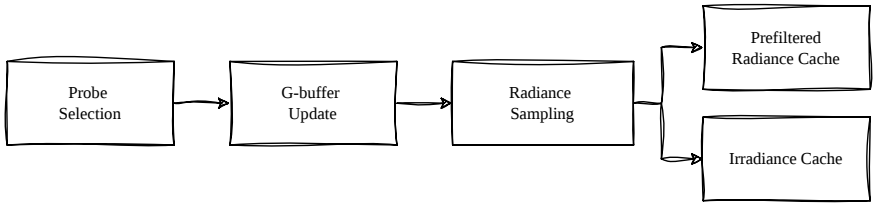


Figure 8: Schematic summary illustrating the major steps comprising the probe volume update procedure.

The following sections provide more details for each part of this process. We begin with Section 4.1 describing the way each probe spreads its ray samples across the entire spherical domain to achieve a uniform sampling pattern under a fixed per-probe ray budget. Section 4.2 then briefly details how lighting is computed at each hit position and how ray misses are handled. We continue by detailing the subsequent processing of those raw

radiance samples to derive the irradiance and prefiltered radiance caches stored at each probe location. Section 4.3 then identifies different criteria to help selecting which probes require an update, as well as discussing different update modes. Finally, Section 4.4 discusses our solutions to mitigate light and shadow leaks by handling both visibility and occlusion.

4.1 Generating Rays

To capture the surrounding illumination at a probe position $\mathbf{x} \in \mathbb{R}^3$, the probe must sample the incoming radiance field $L_i(\mathbf{x}, \omega_i)$ across the full spherical domain. However, evaluating this continuous spherical integral analytically is intractable for general environments. Therefore, we approximate the incoming radiance field numerically using Quasi-Monte Carlo methods, as they exhibit a faster convergence rate compared to that of classic Monte Carlo. Additionally, as discussed in Section 2.1, QMC replaces random sampling with deterministic low-discrepancy sequences which may be precomputed and reused, leading to overall better performance.

Numerous LDS have been used in computer graphics to solve the rendering equation integral over the spherical domain, such as the Sobol's (0,2)-sequence [46], blue noise-based sequences [24] and many others. Among these, we chose the spherical Fibonacci (SF) sequence for its native spherical construction, as opposed to many other LDS, which are generated on the unit square and later mapped to the sphere, resulting in distortion as the spherical topology is not taken into consideration. Furthermore, SF achieves comparable image quality with other LDS but with fewer samples, thanks to its uniform coverage of the spherical domain. For a comprehensive analysis and comparison with other LDS, we refer the reader to the work by Marques et al.[31].

The coordinates of an N-point SF set are given by:

$$\begin{cases} \theta_i = \arccos\left(1 - \frac{2i+1}{N}\right) \\ \phi_i = 2i\pi\Phi^{-1} \end{cases} \qquad \text{with } 0 \leq i < N \tag{4.1}$$

These coordinates are expressed in spherical form on a unit sphere, where θ_i denotes the polar angle and ϕ_i denotes the azimuthal angle of the i-th point. Here, Φ corresponds to the golden ratio defined as $\frac{1+\sqrt{5}}{2}$. A visual example is provided in Figure 9.

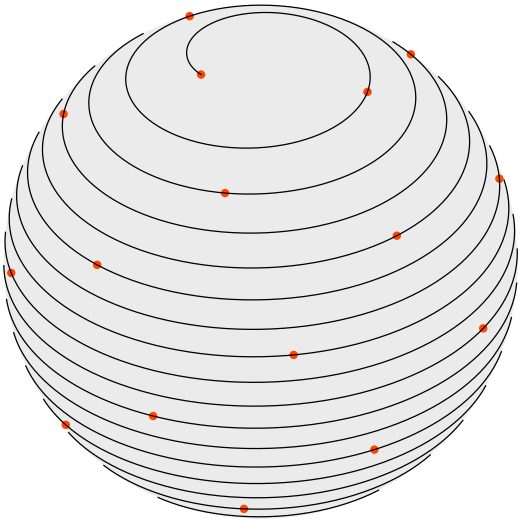


Figure 9: A SF sequence of 30 sample points are generated over the unit sphere following Equation 4.1. These sample points are then used to compute the set of directions toward which the rays are cast.

Because modern hardware ray tracing APIs operate on three-dimensional Cartesian vectors, the spherical coordinates (θ_i, ϕ_i) are converted into unit direction vectors $\mathbf{d}_i \in \mathbb{R}^3$. For a fixed ray budget N, this base set of directions is precomputed once and reused throughout execution. However, casting rays toward identical directions for each probe causes neighboring probes to sample at identical angles, creating structured grid artifacts and preventing temporal accumulation from gathering new directional coverage over time. To eliminate these issues, we follow the DDGI approach [29] and multiply the base directions by a per-probe stochastic rotation matrix $\mathbf{R}$, generated on-the-fly during the updating process.

Finally, by sharing the probe origin $\mathbf{x}$, the set of N parametric rays traced from a single probe is defined as:

$$\mathbf{r}_i(t) = \mathbf{x} + t \cdot (\mathbf{R}\mathbf{d}_i), \quad 0 \leq t \leq t_{\max} \quad 0 \leq i < N \tag{4.2}$$

Here, the maximum ray distance $t_{\max}$ is dynamically scaled according to the probe's cascade level, increasing the coverage distance for higher cascades.

Once the set of sampling directions is established, we proceed by dispatching the rays into the scene. In the following section, we thus examine

how the scene is actually sampled and how those raw samples are processed into stable data.

4.2 Radiance Sampling

During the ray tracing stage, we proceed by testing whether each ray intersects any scene geometry. If an intersection occurs, the surface attributes at the hit point, such as world position, normal, and material properties, are all stored in the probe G-buffer cache (Section 3.2). This caching mechanism decouples ray-scene traversal from lighting evaluation, allowing dynamic lighting to be re-evaluated on the cached surface data, without the need to re-trace any ray. We leverage this decoupling in Section 4.3 to devise partial updates that minimize ray tracing overhead while maintaining responsive lighting updates. Furthermore, it is in this stage that the depth map containing the first and second depth moments is derived from each ray hit distance and stored at each probe. With the surface attributes cached, both direct lighting from analytical light sources and indirect diffuse lighting are then evaluated at the hit position. Indirect diffuse lighting is obtained by sampling the probe volume irradiance cache from the previous frame, thus resulting in multi-bounce global illumination.

If no intersection occurs, we use the previously defined maximum distance $t_{\max}$ to check whether the ray endpoint still falls inside the probe volume bounding box. If so, we sample the prefiltered radiance cache from the previous frame and use that as an approximation of the incoming radiance at that point. Otherwise, we fall back to sampling the far-field environment model (e.g., a skybox or HDR map).

In addition to lighting evaluation, the ray tracing stage also performs a preliminary occlusion check for each probe by keeping count of how many of its dispatched rays terminate on a back-facing surface. If the overall proportion of back-face hits exceeds a predefined threshold, the probe is assumed to be embedded within solid geometry and it is marked as inactive. Temporarily deactivating these internal probes prevents them from influencing the interpolation when sampling the probe volume, thereby eliminating shadows leaking in adjacent visible regions. Furthermore, it is in this stage that the depth map containing the first and second depth moments is derived from each ray hit distance and stored at each probe. Other aspects about visibility and occlusion handling are provided in Section 4.4.

Once incoming radiance has been evaluated either through explicit computation or sampling, rather than storing the raw, noisy samples directly, the framework filters them on-the-fly to produce more stable diffuse and

specular indirect lighting caches, adapting the environment map filtering principles established in Section 2.2. While classical IBL precomputes these filtering integrals offline for an infinitely distant, spatially invariant light field, our framework relaxes this assumption by treating each probe as a localized, position-dependent environment map. By filtering its gathered QMC samples in real-time, each probe transforms high-variance evaluations into stable signals before storing them into its irradiance and prefiltered radiance caches. Furthermore, to store these continuous spherical functions in 2D textures, we use octahedral projection [8], as introduced in Section 3.2. In practice, this filtering pass may be implemented via a single compute shader where each GPU thread processes a single texel within a probe's octahedral tile. The filtering processes to derive the irradiance and prefiltered radiance caches from the raw radiance samples are both discussed next.

4.2.1 Irradiance Cache Derivation

To compute the indirect diffuse lighting stored in the probe's irradiance cache, each texel is assigned a compute thread which must evaluate and store the incoming incident irradiance $E(\mathbf{n})$, along the surface normal $\mathbf{n}$. The normal $\mathbf{n}$ must be reconstructed (and re-normalized) by each thread through the inverse projection of the octahedral UV coordinates of the texel's center:

$$\mathbf{n} = \mathbf{o}^{-1}(u, v) \tag{4.3}$$

where $\mathbf{o}^{-1}$ denotes the inverse octahedral projection function [8]. Intuitively, each texel of the irradiance cache stores the incident irradiance arriving at a hypothetical surface located at the probe center and oriented along $\mathbf{n}$, with $\mathbf{n}$ spanning the entire spherical domain S^2 across the octahedral map.

Classically, continuous irradiance $E(\mathbf{n})$ is defined as in Equation 2.14 as an integral of incident radiance $L_i(\omega_i)$ over the upper hemisphere Ω. However, because our probes dispatch rays uniformly across the entire unit sphere rather than just an oriented hemisphere, we further extend the integration domain to cover S^2. To filter out non-contributing samples, we replace the standard cosine term with a clamped dot product, zeroing-out incident light coming from behind the hypothetical surface orientation $\mathbf{n}$:

$$E(\mathbf{n}) = \int_{\omega_i \in \mathbb{S}^2} L_i(\omega_i) \cdot \max(0, \omega_i \cdot \mathbf{n}) \, \mathrm{d}\omega_i \tag{4.4}$$

By applying the standard MC estimator from Equation 2.1, we approximate continuous incoming radiance $L_i(\omega_i)$ using the finite set of raw radiance

samples $L_i(\mathbf{d}_i)$ previously gathered during the ray tracing pass, thus we set the MC integrand to $f(\mathbf{d}_i) = L_i(\mathbf{d}_i) \cdot \max(0, \mathbf{d}_i \cdot \mathbf{n})$. Furthermore, since the N rays are uniformly distributed over the 4π steradians of the unit sphere, the probability density function must be constant across all directions, thus yielding $p(\mathbf{d}_i) = \frac{1}{4\pi}$. Substituting both $f(\mathbf{d}_i)$ and $p(\mathbf{d}_i)$ into the estimator gives the following discrete MC approximation:

$$E(\mathbf{n}) \approx \frac{1}{N} \sum_{i=1}^{N} \frac{L_i(\mathbf{d}_i) \cdot \max(0, \mathbf{d}_i \cdot \mathbf{n})}{\frac{1}{4\pi}} = \frac{4\pi}{N} \sum_{i=1}^{N} L_i(\mathbf{d}_i) \cdot \max(0, \mathbf{d}_i \cdot \mathbf{n}) \quad (4.5)$$

This final summation thus represents the computational work that must be executed in parallel, by each compute thread, to filter the raw radiance samples to produce the final probe irradiance cache.

4.2.2 Prefiltered Radiance Cache Derivation

A similar process is concurrently executed within the same compute shader to derive the probe prefiltered radiance cache. The goal of this filtering operation is to smooth the previously gathered raw radiance samples into a stable cache, which can later be queried as a fallback, preserving incoming directional radiance.

Each texel is strictly associated with a corresponding target direction $\mathbf{s}$, once again decoded via inverse octahedral projection of the texture's UV-coordinates:

$$\mathbf{s} = \mathbf{o}^{-1}(u, v) \quad (4.6)$$

To filter the incoming radiance signal, we follow the approach presented by LeBlanc et al. [27] and employ the following clamped cosine filter kernel:

$$K(\mathbf{d}_i, \mathbf{s}) = \left[\max\left(0, \frac{\mathbf{d}_i \cdot \mathbf{s} - c}{1 - c}\right) \right]^p \quad (4.7)$$

where $\mathbf{d}_i \cdot \mathbf{s}$ represents the cosine of the angle between the raw radiance sample direction and the target direction, $c = 0.75$ establishes an angular cutoff at approximately 41.4, and $p = 4$ controls the smooth falloff.

Extending once again the integration domain to the entire unit sphere $\mathbb{S}^2$, the continuous prefiltered radiance relative to target direction $\mathbf{s}$ is defined as:

$$L_{\text{prefiltered}}(\mathbf{s}) = \int_{\omega_i \in \mathbb{S}^2} L_i(\omega_i)\, K(\omega_i, \mathbf{s})\, \mathrm{d}\omega_i \quad (4.8)$$

By applying the standard Monte Carlo estimator to Equation 4.8 and accounting for the uniform spherical sampling by setting $p(\mathbf{d}_i) = \frac{1}{4\pi}$, we obtain the unnormalized estimator over N discrete ray directions $\mathbf{d}_i$:

$$L_{\text{prefiltered}}(\mathbf{s}) \approx \frac{4\pi}{N} \sum_{i=1}^{N} L_i(\mathbf{d}_i)\, K(\mathbf{d}_i, \mathbf{s}) \tag{4.9}$$

Because the kernel function $K(\omega_i, \mathbf{s})$ is unnormalized, evaluating Equation 4.9 over a finite set of samples causes the overall accumulated weight to deviate from unity, introducing severe brightness artifacts depending on the local sample distribution. Following Owen [37], we address this by formulating our estimator as a self-normalized ratio estimator, dividing the unnormalized sum by the total accumulated sample weights:

$$L_{\text{prefiltered}}(\mathbf{s}) \approx \frac{\frac{4\pi}{N} \sum_{i=1}^{N} L_i(\mathbf{d}_i)\, K(\mathbf{d}_i, \mathbf{s})}{\frac{4\pi}{N} \sum_{i=1}^{N} K(\mathbf{d}_i, \mathbf{s})} \tag{4.10}$$

Canceling out the global factor $\frac{4\pi}{N}$ from both the numerator and denominator gives the final discrete ratio estimator evaluated by our compute shader:

$$L_{\text{prefiltered}}(\mathbf{s}) \approx \frac{\sum_{i=1}^{N} L_i(\mathbf{d}_i)\, K(\mathbf{d}_i, \mathbf{s})}{\sum_{i=1}^{N} K(\mathbf{d}_i, \mathbf{s})} \tag{4.11}$$

By convolving discrete ray samples into a continuous directional field, the prefiltered radiance cache thus acts as a more stable and reliable estimate of incoming radiance.

Once derived, the irradiance and prefiltered radiance caches are both stored within their corresponding atlases. To suppress low-sample Monte Carlo noise and eliminate temporal flickering, the irradiance cache makes further use of exponential hysteresis blending across frames [29]. The same idea is also applied for each probe's depth map.

Regarding storage, to account for the two signals different spatial frequencies, both caches use different interior resolutions. Prefiltered radiance requires an higher resolution, such as 10×10, to better preserve directional detail, whereas a compact 5×5 resolution suffices for the smooth irradiance signal [27]. To eliminate internal seam artifacts across octahedral boundaries and prevent inter-probe color bleeding between adjacent tiles in each global atlas during bilinear filtering, each texture is extended with a single-pixel guard band by reflecting interior edge texels across the octahedral boundary [14]. This border expands the total maps sizes to 12×12 and 7×7 texels,

respectively. Packed into the probe volume texture atlases, both caches are ready for being efficiently sampled during the later shading pass.

4.3 Update Policies

Naively updating each light probe every frame is far too expensive in a real-time context, where ensuring a constant, interactive frame rate typically requires rendering and displaying an image in less than 30 milliseconds. A way to do so is to establish a per-frame ray budget (tailoring it to the specific target hardware) and to devise a set of policies in order to better partition that budget to just a subset of the total probes. We consequently organize the discussion on these update policies around two fundamental key aspects. The first aspect is *probe selection*, which encompasses a set of criteria based on spatial and temporal heuristics, to determine which probes should be selected and updated. The second aspect are *update modes*, where we detail the specific modalities, with different costs, used to update a single light probe. Note that these policies only take into account light probes currently marked as active, as inactive (i.e. occluded) probes are assigned zero rays and entirely excluded from the scheduling process (Section 4.4).

We begin the discussion by first addressing the probe selection mechanism. To select which probes must be updated for the current frame, we rely on a priority queue data structure and a set of criteria that should be considered when assigning the priority of each probe. The highest update priority should be assigned to the newly spawned probes. That is, probes which just appeared on the leading-edge of the probe volume as a consequence of camera movement (see Figure 4). In fact, these new probes no longer hold any meaningful data about the radiance field at their new position, thus requiring to be fully updated. Furthermore, if temporal hysteresis is in use (see Section 4.2), the hysteresis coefficient should be set to zero to completely overwrite the old stored data and thus avoiding the presence of ghosting artifacts caused by persistent data.

Once part of the ray budget is allocated to this initial set of probes (usually $0 - 50\%$ of the initial ray budget [27]), the remaining rays are further distributed among the other active probes by considering additional heuristics. For example, probes located closer to the camera may be prioritized next, as their radiance directly impacts a larger proportion of screen-space pixels. Similarly, probes positioned near complex geometric surfaces should receive an higher priority than those in open spaces, since proximity to geometry introduces high-frequency lighting and occlusion signals. However, because spatial heuristics tend to continually favor probes located either

near the camera or geometry, by relying on them alone, we risk to always update the same subset of probes every frame. To prevent probe starvation in the priority queue, we propose an additional temporal heuristic that tracks the time elapsed since a probe latest update, progressively boosting its priority until a new one occurs.

Having established the way we select probes for re-evaluation, we next turn to the update modalities. Inspired by recent industry solutions [27], we categorize these update modes into three tiers based on their execution cost, where an higher execution cost requires more rays, computation and time for it to be executed, but yields an higher visual quality results. The highest tier corresponds to a fully ray-traced update executed at full resolution, encompassing both scene traversal and prefiltered radiance and irradiance cache computations, thus updating both cached geometry and lighting. This pass is intended for high-priority targets, such as the newly spawned probes or probes located near the camera. The mid tier update mode is instead meant to amortize scene traversal overhead in distant regions by still executing a fully ray-traced update, but this time at quarter resolution. Updating probes with a reduced ray count can in fact reduce the total ray tracing costs by up to 40% [27] while still preserving low-frequency directional details. Lastly, within certain specific contexts, for example when only dynamic illumination changes (such as with moving light sources or time-of-day transitions) while both geometry and probe position remain instead static, the probe can only be partially updated. By partial update, we mean exploiting the cached surface attributes from the per-probe G-buffer cache to avoid executing the expensive ray tracing pass entirely, thus resulting in no rays cast. Instead, we can just re-evaluate lighting at those cached hit positions and then update the prefiltered radiance and irradiance cache from the result.

A common situation in interactive environments that should be addressed while updating each probe, involves abrupt illumination shifts caused by user input, such as the player turning the lights on or off. To maintain visual responsiveness in this scenario, active probes awaiting an update should undergo a dynamic illumination scaling. For example, we can scale prefiltered radiance and irradiance values by the ratio between the current and previous frame average luminance, approximated by considering only the dominant light sources, thus reducing temporal lag and ghosting artifacts without consuming additional ray budget.

In conclusion, while in this section we presented general hints to help better allocate the ray budget, we emphasize that the specific policies and update modes should always be designed and tailored based on the specific needs of the project at hand, taking different factors into consideration such

as target hardware and platform, scene scope and complexity and so on.

4.4 Visibility and Occlusion Handling

When using a probe-based approach to solve global illumination, naively interpolating light data from neighboring probes without taking visibility and geometric occlusion into account often results in severe visual artifacts. For instance, two well-documented, unwanted effects are *light leaking* and *shadow leaking*, both resulting from sampling occluded probes data, thereby causing either light to spill into dark regions or shadows to bleed into bright ones.

To address these problems, we separate the discussion in two parts. First, we address *visibility*, which determines whether valid light interactions can occur between a probe and the surface we are shading based on the latter's orientation. We account for this by introducing a technique called *wrap shading* to smoothly reduce contributions coming from probes located behind the shaded surface. This results in the computation of one of the weights, w_{vis}, used when interpolating sampled data from the probe volume as detailed in Section 3.4. Next, we discuss *occlusion*, which accounts for opaque geometric surfaces placed in-between the probe and surface, thus interfering with light transport. This is roughly addressed during the ray tracing pass by disabling probes embedded inside geometry which result in too many back face hits. Then later, when sampling from the probe volume, we apply a more accurate method using depth mean and variance to probabilistically estimate the presence of obstacles through Chebyshev's test to then set an additional interpolation weight w_{occ}.

4.4.1 Visibility and Backface Filtering

Visibility evaluation determines whether a probe is able to illuminate a surface point without considering intervening physical obstacles. In fact, probes positioned behind the hemisphere oriented along surface normal $\mathbf{n}$ cannot directly contribute to the front face lighting of an arbitrary point $\mathbf{x}$ belonging to that surface. To check if that is the case, we measure directional alignment between the point surface normal $\mathbf{n}$ and the unit direction vector $\mathbf{d}$ pointing toward probe $\mathbf{p}$ and defined as:

$$\mathbf{d} = \frac{\mathbf{p} - \mathbf{x}}{\|\mathbf{p} - \mathbf{x}\|} \tag{4.12}$$

Since evaluating directional alignment by using a standard cosine factor yields an abrupt change in probe contributions at grazing angles, Majercik et al. [29], in their DDGI implementation, suggest a more complex function to compute a softer attenuation. This function is called wrap shading and was originally formulated by Sloan et al. [45] as a simple approximation for subsurface scattering, simulating the response of incident light on a back lit surface. By using the wrap shading function, the attenuation weight is formally defined as:

$$w_{\text{vis}}(\theta) = \left(\frac{\cos\theta + a}{1 + a}\right)^{1+a} \tag{4.13}$$

where θ is the angle between the point surface normal $\mathbf{n}$ and the probe direction vector $\mathbf{d}$, and $a \in [0, 1]$ specifies the smooth wrapping factor. The DDGI implementation sets $a = 1$, thus resulting in:

$$w_{\text{vis}}(\theta) = \left(\frac{\cos\theta + 1}{2}\right)^{2} \tag{4.14}$$

Instead of hard-clamping at 90°, this function smoothly attenuates probe contributions past the surface tangent plane, reaching absolute zero at $\theta = 180°$. A comparison between this function and the standard cosine factor is illustrated in Figure 10.

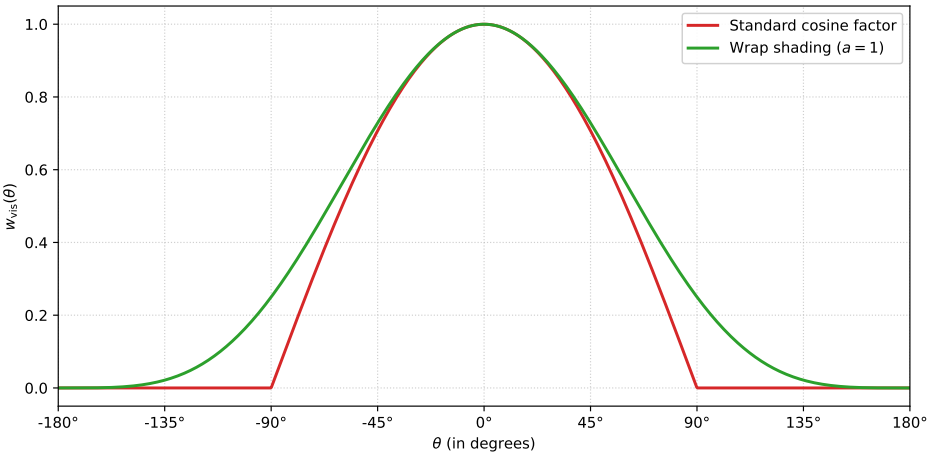


Figure 10: Comparison between two different visibility attenuation functions. The red curve shows the standard Lambertian cosine factor while the green curve illustrate the wrap shading function as implemented in DDGI [29]. Notice their behaviors at grazing angles: while wrap shading smoothly decreases light contribution, the standard cosine factor results in an abrupt cut instead.

This method successfully manages to smooth shading discontinuities at grazing angles while also suppressing back face light bleeding.

4.4.2 Occlusion Handling and Surface Biasing

With wrap shading accounting for probe-surface visibility, we now turn to the evaluation of occlusion.

We already mentioned an initial heuristic used while updating the probe volume. In fact, during that stage, each probe casts a set of rays into the scene to collect both radiance and depth data. If the probe is positioned inside either static or dynamic geometry, a high proportion of its update rays will end up hitting back facing surfaces. By keeping count of how many back faces are hit and by comparing this value with a chosen threshold, we choose whether to consider the probe occluded or not and mark it as inactive or active, respectively.

By following the DDGI approach, we also present an additional and more rigorous method originally based within variance shadow mapping literature. Variance shadow mapping is a shadow mapping technique introduced in 2006 by Donnelly and Lauritzen [12]. It differs from classical shadow mapping [51]

by storing the first two depth moments instead of raw depth values, thus replacing the standard binary depth test with a probabilistic one which also allows for the correct usage of standard texture filtering and anti-aliasing techniques on depth data, something that previously yielded incorrect values with classic shadow maps.

Recalling from Section 3.2, each probe stores the first and second moments of the hit distance r, $M_1 = \mathbb{E}[r]$ and $M_2 = \mathbb{E}[r^2]$, directly within its two-channel, octahedral depth texture, which may be allocated at the same resolution as the prefiltered radiance cache (12×12 texels, including the gutter border). During shading, we then sample M_1 and M_2 along $-\mathbf{d}$ (i.e., the unit vector pointing from the probe to the surface) to retrieve both the depth mean and mean square to compute the depth variance:

$$\begin{aligned} \mu &= M_1, \\ \sigma^2 &= M_2 - M_1^2 \end{aligned} \tag{4.15}$$

The variance σ^2 can be interpreted as a quantitative measure of the depth distribution's spread across the sampled region. A variance near zero indicates a single surface at uniform depth (e.g. a wall), whereas a high variance indicates the presence of depth discontinuities, such as in a complex scenes or with complex geometry being present within the region. Because it bounds how much of the distribution can lie far from the mean depth μ, it allows us to constrain the fraction of the depth distribution that is located beyond a given distance. In probability theory, this relationship is formalized by the one-tailed version of Chebyshev's inequality. For a random depth variable r with mean μ and variance σ^2, the probability that a depth sample exceeds a distance $t > \mu$ is upper-bounded by:

$$P(r \geq t) \leq \frac{\sigma^2}{\sigma^2 + (t - \mu)^2} \tag{4.16}$$

In this context, if we set $t = \| - \mathbf{d}\|$, that is, the distance between the probe and the surface point $\mathbf{x}$, $P(r \geq t)$ then represents the upper-bound on the probability that the geometry stored in the probe depth map lies farther away than the surface point itself. In other words, if the probability $P(r \geq t)$ is high, most of the depth distribution lies farther away than $\mathbf{x}$, meaning that the surrounding geometry is located behind the surface point, thus resulting in an unoccluded path. This concept was first introduced with variance shadow maps as an alternative and probabilistic version of the standard depth test. Here, we use the same idea to define the occlusion weight $w_{\text{occ}}(t)$ as:

$$w_{\text{occ}}(t) = \begin{cases} 1.0 & \text{if } t \leq \mu, \\ \dfrac{\sigma^2}{\sigma^2 + (t-\mu)^2} & \text{if } t > \mu. \end{cases} \tag{4.17}$$

However, evaluating Chebyshev bounds directly may cause self-shadowing artifacts. To resolve this, we offset the evaluation position away from the surface before calculating distance t [35]:

$$\mathbf{b} = k\,(0.8\mathbf{n} + 0.3\mathbf{v}) \tag{4.18}$$

where $\mathbf{v}$ is the view vector toward the camera and $k \in [0, 1]$ acts as a user-defined scaling multiplier. Evaluating Chebyshev queries at the biased point $\mathbf{x}_{\text{biased}} = \mathbf{x}+\mathbf{b}$ reduces the presence of self-shadowing artifacts, though carefully tuning the scaling factor k is required to avoid an exaggerated bias resulting in light bleeding through thin walls. While this method works well in many situations, it is not guaranteed to be an accurate approximation. In practice this is only a problem for scenes with an high depth and geometrical complexity, while scenes with a low complexity such as open areas produce non-existent or negligible artifacts.

In summary, combining wrap shading with normal-biased Chebyshev occlusion tests provides a solid, real-time solution to light and shadow leaks artifacts for most scenes. These weights, are computed and integrated with the standard trilinear interpolation weight during probe volume sampling as described in Section 3.4.

Chapter 5

Surface Shading

So far, Chapter 3 introduced the probe volume data structure, describing its core properties and sampling methods, while Chapter 4 detailed the update process for each individual probe. This chapter thus serves as an overall conclusion to everything we have built so far, detailing how the probe volume interfaces with the rest of the rendering pipeline to compute indirect illumination.

We begin by first rasterizing the scene from the point of view of the camera by following a standard deferred rendering pipeline [49]. Within the shader associated to each visible surface point $\mathbf{x}$, reconstructed from the G-buffer, we proceed by computing direct illumination from all contributing punctual and area lights using the chosen local illumination model. Next, both indirect diffuse and specular illumination are computed independently by shooting secondary rays (one ray each). Details on how these rays are dispatched into the scene and indirect illumination is computed are discussed in Section 5.1 and Section 5.2.

In order to accelerate intersection testing, following the approach used by LeBlanc and Conte in the production of Assassin's Creed Shadows [27], screen-space ray marching may be additionally performed prior to standard bounding volume hierarchy (BVH) traversal, as the former is generally faster. Rays that fail to intersect any geometry during the screen-space pre-pass (e.g. by exiting the screen boundaries), are re-cast into the scene during the subsequent world-space ray tracing pass.

5.1 Indirect Diffuse Illumination

The ray direction for evaluating indirect diffuse illumination is importance-sampled according to the surface material's chosen diffuse BRDF. In other words, for each surface point $\mathbf{x}$, the secondary ray direction ω_{diff} is selected within its upper hemisphere $\Omega(\mathbf{n})$ (with $\mathbf{n}$ being the surface normal in $\mathbf{x}$) by sampling a probability density function $p(\omega)$ that depends on the chosen diffuse model. For instance, importance-sampling the standard Lambertian BRDF, also known as cosine-weighted sampling, uses the following PDF:

$$p(\omega) = \frac{\cos\theta}{\pi} \tag{5.1}$$

with θ being the angle between ω and $\mathbf{n}$ [38].

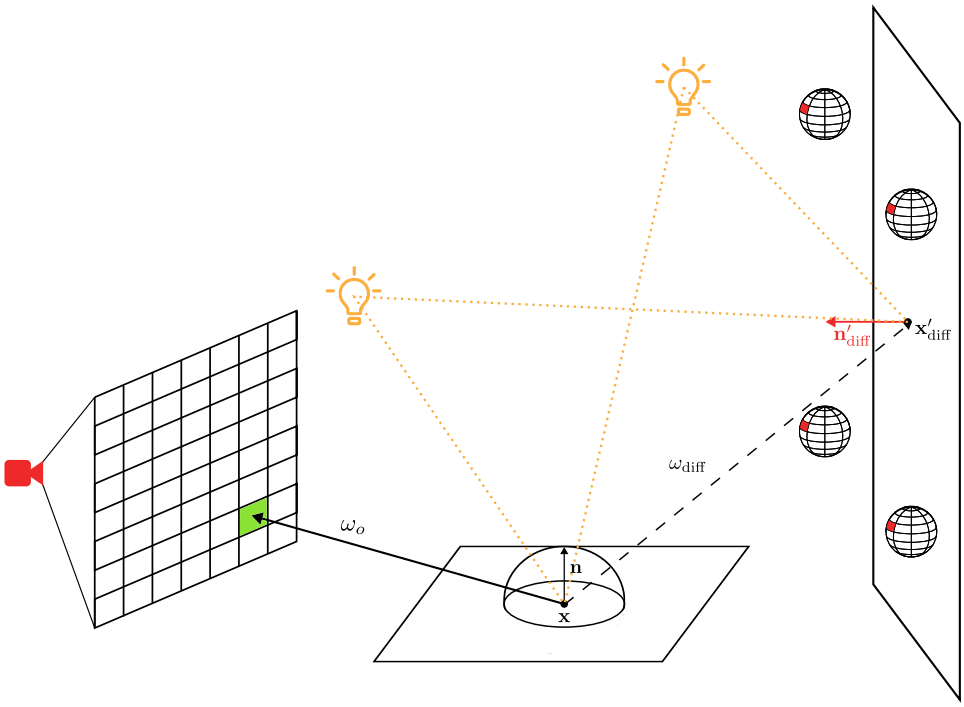


Figure 11: Schematic representation of all the elements involved in the shading of surface point $\mathbf{x}$. Here, to evaluate the indirect diffuse illumination, a ray is cast along the importance-sampled direction ω_{diff}. Direct and indirect illumination are then computed at the hit position $\mathbf{x}'_{\text{diff}}$. The surface normal $\mathbf{n}'_{\text{diff}}$ (shown in red) is used to sample the corresponding irradiance cache texel (also shown in red) from the eight nearest probes, serving as indirect lighting at this secondary point.

Once the ray has been dispatched along ω_{diff}, we check for any intersection. If an intersection occurs at a hit point $\mathbf{x}'_{\text{diff}}$, we proceed by evaluating both direct illumination (as performed for the original surface point $\mathbf{x}$) and indirect diffuse illumination. We deliberately omit computing indirect specular lighting at this secondary hit point as the trade-off between added computational complexity and perceptual visual difference is minimal. To evaluate indirect diffuse illumination we sample the probe volume at surface

point $\mathbf{x}'_{\text{diff}}$, retrieving and interpolating the irradiance cache values stored at the eight nearest probes, as detailed in Section 3.4.

If, otherwise, the ray fails to intersect any surface, we further check if its end point still falls within the probe volume bounds. If it does, we sample the prefiltered radiance cache instead, using ω_{diff} as the sampling direction to approximate the radiance field at that point. This is done, in contrast with sampling the irradiance cache, as the prefiltered radiance cache retains a greater angular resolution. If, instead, the end point falls outside the AABB delimiting the probe volume, we fall back to sampling a low-frequency irradiance environment map (e.g. a preconvolved skybox).

Once evaluated, combining both direct and indirect lighting at $\mathbf{x}'_{\text{diff}}$ yields the incoming radiance $L_i(\mathbf{x}, \omega_{\text{diff}})$ at the original surface point $\mathbf{x}$, thus serving as the single-sample indirect diffuse contribution.

While computing this indirect diffuse contribution for every pixel using just a single ray sample per pixel (1 SPP) is necessary to keep execution times within real-time budgets, this low sample rate will inevitably introduce stochastic noise into the signal. Naively denoising these raw RGB radiance values though, does not take any directional information into account, causing the resulting denoised indirect diffuse lighting to appear completely flat and unresponsive to surface details (e.g. achieved through normal or bump mapping [4]) [27]. To solve this issue, before feeding the 1 SPP incoming radiance signal $L_i(\mathbf{x}, \omega_{\text{diff}})$ to a denoiser, we first encode it into low-order spherical harmonics [2, 39]. Spherical harmonics (SH) are orthonormal basis functions defined on the sphere that can be viewed as the spherical analogue of the classical Fourier basis on the unit circle, thus enabling the representation of arbitrary spherical functions[8]. By using SH encoding, we successfully manage to hold onto directional data during the denoising process, preserving more high-frequency details and thus producing more qualitative results [27].

A further improvement proposed by LeBlanc et al. is converting the signal from the linear RGB color space into the YCoCg color space [30] prior to SH encoding. This intermediate conversion serves as an improvement over the heavy interdependency between the standard RGB channels. In fact, when light intensity changes both the red, green, and blue channels scale together. By transforming radiance to YCoCg we decorrelate the signal by separating luminance (Y) from orange and green chrominance (Co and Cg) through a fast linear matrix transform. This also aligns with the higher

[8] A comprehensive and practical introduction to their application in real-time computer graphics is provided by Sloan [44].

sensitivity of human visual perception to variations in luminance rather than chrominance. While similar color spaces such as YCbCr also exist, YCoCg is preferred in real-time graphics as its conversion relies exclusively on power-of-two coefficients and additions rather than arbitrary floating-point multiplications, resulting in a faster conversion [30].

Once $L_i(\mathbf{x}, \omega_{\text{diff}})$ has been converted, we then leverage the perceptual separation of luminance and chrominance by encoding them with a different number of SH coefficients. More specifically, to retain directional information, we encode luminance using a complete first-order SH expansion, comprising both the constant $l = 0$ term and the three linear $l = 1$ coefficients. Orange (Co) and green (Cg) chrominance are instead restricted to just zero-order ($l = 0$) constant terms (one for each channel). Formally, incoming radiance $L_i(\mathbf{x}, \omega)$ is projected into SH space via the following spherical integral:

$$c_{lm}(\mathbf{x}) = \int_{\omega \in \mathbb{S}^2} L_i(\mathbf{x}, \omega)\, Y_{lm}(\omega)\, d\omega \tag{5.2}$$

where $Y_{lm}(\omega)$ denotes the real spherical harmonic basis function of degree l and order m and $c_{lm}(\mathbf{x})$ represents the corresponding SH coefficient. In our context, this continuous integral is numerically evaluated using just our single discrete ray sample along direction ω_{diff}. The six SH coefficients are thus computed by projecting the radiance values onto their corresponding SH basis functions evaluated at direction ω_{diff}:

$$\mathbf{c}_{\text{noisy}}(\mathbf{x}) = \begin{pmatrix} L_i^Y(\mathbf{x}, \omega_{\text{diff}})\, Y_{0,0}(\omega_{\text{diff}}) \\ L_i^Y(\mathbf{x}, \omega_{\text{diff}})\, Y_{1,-1}(\omega_{\text{diff}}) \\ L_i^Y(\mathbf{x}, \omega_{\text{diff}})\, Y_{1,0}(\omega_{\text{diff}}) \\ L_i^Y(\mathbf{x}, \omega_{\text{diff}})\, Y_{1,1}(\omega_{\text{diff}}) \\ L_i^{Co}(\mathbf{x}, \omega_{\text{diff}})\, Y_{0,0}(\omega_{\text{diff}}) \\ L_i^{Cg}(\mathbf{x}, \omega_{\text{diff}})\, Y_{0,0}(\omega_{\text{diff}}) \end{pmatrix} \tag{5.3}$$

where $L_i^Y(\mathbf{x}, \omega_{\text{diff}})$, $L_i^{Co}(\mathbf{x}, \omega_{\text{diff}})$, and $L_i^{Cg}(\mathbf{x}, \omega_{\text{diff}})$ denote the luminance, orange chrominance, and green chrominance components of the incoming indirect diffuse radiance $L_i(\mathbf{x}, \omega_{\text{diff}})$, respectively, while the first four real spherical harmonic basis functions are numerically given by [39]:

$$Y_{0,0}(\omega) = 0.282095, \quad \text{and} \quad \begin{pmatrix} Y_{1,-1}(\omega) \\ Y_{1,0}(\omega) \\ Y_{1,1}(\omega) \end{pmatrix} = 0.488603\, \omega \tag{5.4}$$

Notice that converting the signal to the YCoCg color space allows us to isolate directional data exclusively within the luminance component while omitting it for chrominance, reducing the representation to just six SH coefficients as opposed to the twelve required for a full first-order RGB encoding where all the three, highly correlated channels must carry directional detail.

Once each pixel's incoming indirect diffuse illumination is successfully encoded in SH space, spatio-temporal denoising is then executed by leveraging modern denoising strategies such as spatiotemporal variance-guided filtering (SVGF) [41], which is specifically designed to reconstruct clean illumination signals from noisy 1 SPP inputs while also ensuring interactive frame rates[9]. On the practical side, the NVIDIA Real-Time Denoisers (NRD) library is an API-agnostic framework offering three different denoisers (REBLUR, RELAX and SIGMA) to reconstruct 1 SPP signals in real-time. Furthermore, it includes a SH-space filtering modes that achieves output quality comparable to modern AI-driven technologies like DLSS Ray Reconstruction (DLSS-RR), serving as a reliable, non-AI practical solution [34].

Following spatio-temporal filtering, the resulting denoised SH coefficients vector $\mathbf{c}_{\text{denoised}}(\mathbf{x}) = \begin{pmatrix} c_1 & c_2 & c_3 & c_4 & c_5 & c_6 \end{pmatrix}^T$ is convolved with the Lambertian cosine kernel and evaluated against the G-buffer surface normal $\mathbf{n}$ to compute the incoming indirect diffuse irradiance for each surface $\mathbf{x}$. Following Ramamoorthi and Hanrahan [39], the reconstructed irradiance $E(\mathbf{n})$ from a set of spherical harmonic coefficients c_{lm} is given by:

$$E(\mathbf{n}) = \sum_{l,m} A_l \, c_{lm} \, Y_{lm}(\mathbf{n}) \tag{5.5}$$

where A_l are the analytical convolution expansion factors which, for the Lambertian cosine kernel, result in the first two factors to be $A_0 = \pi$ and $A_1 = \frac{2\pi}{3}$ (see the work by Ramamoorthi and Hanrahan [39] for the full derivation). By applying Equation 5.5 to $\mathbf{c}_{\text{denoised}}(\mathbf{x})$, we get:

$$\begin{aligned} E^Y(\mathbf{x}, \mathbf{n}) &= A_0 \, Y_{0,0} \, c_1 + A_1 \left(Y_{1,-1}(\mathbf{n}) \, c_2 + Y_{1,0}(\mathbf{n}) \, c_3 + Y_{1,1}(\mathbf{n}) \, c_4 \right) \\ E^{Co}(\mathbf{x}) &= A_0 \, Y_{0,0} \, c_5 \\ E^{Cg}(\mathbf{x}) &= A_0 \, Y_{0,0} \, c_6 \end{aligned} \tag{5.6}$$

[9] A detailed treatment of spatio-temporal denoising algorithms is a complex subject and a matter of its own study. As a deep dive into the underlying topics would be outside the scope of this work, we limit ourselves to treating the denoiser as a modular reconstruction stage within our framework.

An inverse transformation matrix then converts the result back into linear RGB components, yielding the indirect diffuse irradiance $E(\mathbf{x})$ at each surface point $\mathbf{x}$.

Lastly, the reconstructed indirect diffuse irradiance is converted to outgoing radiance reflected towards the camera by evaluating the diffuse BRDF. For instance, for the Lambertian BRDF, we scale it by $\frac{\rho_{\mathbf{x}}}{\pi}$, where $\rho_{\mathbf{x}}$ represents the surface's albedo at $\mathbf{x}$. The indirect diffuse contribution is then combined with the direct diffuse contribution $L_{\text{diff}}^{\text{direct}}(\mathbf{x})$:

$$L_{\text{diff}}^{\text{total}}(\mathbf{x}) = L_{\text{diff}}^{\text{direct}}(\mathbf{x}) + \frac{\rho_{\mathbf{x}}}{\pi} E_{\text{diff}}^{\text{indirect}}(\mathbf{x}) \tag{5.7}$$

Total diffuse radiance $L_{\text{diff}}^{\text{total}}(\mathbf{x})$ thus captures all diffuse direct and indirect lighting reflected from surface point $\mathbf{x}$ towards the camera, completing the first half of the shading process. Next, we turn to direct and indirect specular reflections, thus covering the latter half of the discussion.

5.2 Indirect Specular Illumination

Evaluating indirect specular lighting requires casting again a secondary ray for each surface point $\mathbf{x}$ in the G-buffer. This time, the ray direction ω_{spec} is instead guided by the microfacet normal distribution function which shapes both orientation and spread of the specular lobe. For instance, Heitz [17] provides an efficient method to importance sample the widely adopted GGX distribution [50].

To further reduce computational costs, indirect specular rays are also typically dispatched at a reduced screen resolution, typically half or quarter resolution. Other optimizations include scaling the maximum ray distance dynamically according to surface roughness α. For example, by casting longer rays for smooth, mirror-like surfaces ($\alpha \to 0$) to capture distant reflections and shorter rays for rougher surfaces, we reduce even more scene traversal overhead [27].

Furthermore, we also employ a ray-skipping heuristic that reuses the per-pixel SH radiance representation (which has been already evaluated during the indirect diffuse pass) as fall back indirect specular lighting. In other words, ray dispatch is skipped entirely for extremely rough surfaces (i.e., with a wide specular lobe), or when the importance-sampled secondary ray direction ω_{spec} deviates significantly from the ideal specular reflection direction $\mathbf{r}$ [27]. When a ray is skipped under these conditions, instead of casting rays and querying the probe volume data structure, we directly evaluate the pixel's local diffuse SH representation along the sampled direction ω_{spec},

following the same process as that of Equation 5.6, replacing surface normal $\mathbf{n}$ with ω_{spec}. This optimization makes use of the already-computed indirect diffuse contribution to approximate glossy reflections for rough surfaces, without tracing additional rays.

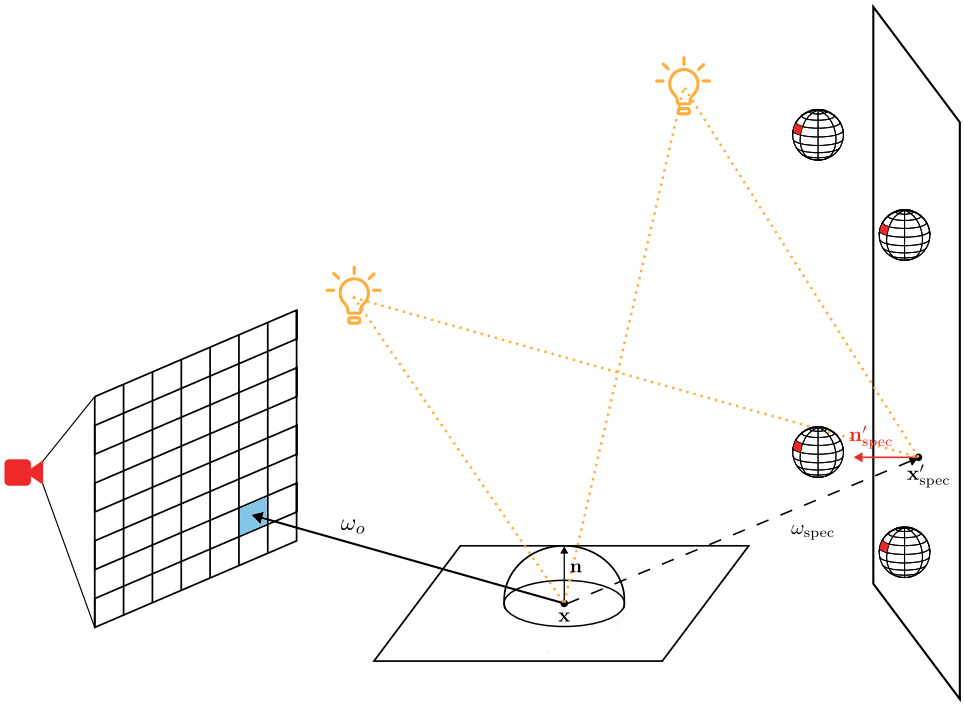


Figure 12: Schematic representation of all the elements involved in the shading of surface point $\mathbf{x}$. Here, to evaluate the indirect specular illumination, a ray is cast along the importance-sampled direction ω_{spec}. Apart from the choice of sampling direction, the overall evaluation procedure mirrors that of the diffuse pass.

For smoother surfaces however, the secondary specular ray is still cast into the scene. When this ray intersects a surface at a secondary hit point $\mathbf{x}'_{\text{spec}}$, we evaluate both direct and indirect diffuse illumination via probe interpolation, following the exact process described in Section 5.1. If, otherwise, the specular ray fails to intersect scene geometry, we again check whether its termination point remains within the probe volume's bounding box. If the ray ends inside the volume, incoming radiance is resolved once again by sampling and interpolating the prefiltered radiance cache from the eight nearest probes. If the ray terminates outside the probe volume bounds,

we instead sample a high-resolution environment map along direction ω_{spec}, serving as a fall back for distant reflections.

In contrast to indirect diffuse lighting, the indirect specular signal is much noisier and difficult to filter. It also cannot be accurately represented with low-order spherical harmonics due to its high-frequency nature. Storing specular reflections in a SH basis would in fact result in both ringing artifacts and blurred reflections. Instead, the noisy specular signal can be denoised directly in linear RGB using the specular pipelines of the NVIDIA NRD library [34]. Better results can also be achieved by using BRDF demodulation as shown by LeBlanc et al. [27] and as detailed in NVIDIA NRD documentation [34].

Subsequently, the denoised indirect specular radiance $L_{\text{spec}}^{\text{indirect}}(\mathbf{x})$ is combined with the analytical direct specular lighting $L_{\text{spec}}^{\text{direct}}(\mathbf{x})$:

$$L_{\text{spec}}^{\text{total}}(\mathbf{x}) = L_{\text{spec}}^{\text{direct}}(\mathbf{x}) + L_{\text{spec}}^{\text{indirect}}(\mathbf{x}) \tag{5.8}$$

At last, adding the total specular radiance $L_{\text{spec}}^{\text{total}}(\mathbf{x})$ with the total diffuse radiance $L_{\text{diff}}^{\text{total}}(\mathbf{x})$ yields the final exitant radiance at surface point $\mathbf{x}$ toward the camera:

$$L_{\text{o}}(\mathbf{x}, \omega_o) = L_{\text{diff}}^{\text{total}}(\mathbf{x}) + L_{\text{spec}}^{\text{total}}(\mathbf{x}) \tag{5.9}$$

This then completes the entire surface shading pass.

Chapter 6

Conclusion

In this work, we presented a real-time, dynamic global illumination framework based on a cascaded probe volume data structure, extending the uniform grid approach of modern solutions such as DDGI [29] and consolidating previously fragmented industry practices into a cohesive and fully documented system.

To achieve this, we formalized the complete lifecycle of the probe volume throughout this work. In Chapter 3, we begin by establishing the main properties of the cascaded grid layout, featuring camera-relative toroidal addressing and four distinct octahedral texture atlases to efficiently store irradiance, prefiltered radiance, distance moments, and cached surface attributes. In Chapter 4, we then discussed its update pipeline, starting from the spherical Fibonacci pattern used by each probe to sample scene radiance and then continuing with the derivation of the irradiance and prefiltered radiance caches. We additionally presented a set of heuristics and considerations to be done when optimizing the distribution of the per-frame ray budget across all active probes. To eliminate light and shadow bleeding, we approach the problem through the lens of visibility and occlusion, combining wrap shading with normal-biased Chebyshev distance moment tests and automatic back face-hit probe deactivation. Finally, in Chapter 5, we have shown how the probe volume integrates within a standard deferred rendering pipeline, detailing how the cache is queried during shading to retrieve both indirect diffuse and specular illumination at any surface point.

We now briefly address some additional improvements and areas for future work. As noted by LeBlanc et al. [27], the interpolation process constitutes roughly half of the total time spent evaluating the indirect lighting cache. In fact, for each secondary hit point, we first require fetching indirect lighting values from the eight surrounding probes, then evaluate both visibility and occlusion to compute the overall interpolation weight and finally, interpolate those values. A major optimization they suggest, consists in limiting sampling only to the single nearest probe, that is, if its interpolation weight is high enough. This is, of course, mainly a shortcut, trading correctness for reduced costs and thus its benefits should be carefully assessed by taking the specific context into consideration.

Regarding visibility, we believe that the usage of an alternative, more physically correct, function to compute the corresponding interpolation weight would constitute an interesting area for future work. This alternative function should still retain some important properties of the wrap shading technique presented in Chapter 4, that is, it should be both fast to compute and smooth within its domain ($[-90°, 90°]$). In contrast to the current approach though, it should follow a more physically correct approach by zeroing probe contributions over a 90° angle. This, of course, should be carefully assessed and compared to the current technique to evaluate its impact on visual fidelity and prevent unwanted rendering artifacts in various situations.

From a practical implementation standpoint, developers deploying this system may also consider the benefit of including complementary screen-space techniques to further augment the solution yielded by our primary probe-based method. More specifically, ambient occlusion techniques such as screen-space ambient occlusion (SSAO) [32, 43] or ground-truth ambient occlusion (GTAO) [19] can compensate for the probe volume's coarse spatial resolution by supplying fine-grained contact shadows. Likewise, screen-space reflection (SSR) [48] algorithms can augment the probe volume's indirect specular component by capturing high-frequency, localized reflections directly from screen-space depth and color buffers. To that end, this framework should be understood as one of many modular components in a graphics engine, one that developers are encouraged to combine and experiment with to build a rendering pipeline tailored to their specific needs.

In conclusion, as we continue to wait for ray tracing to become a viable solution for real-time graphics, it is essential to emphasize the importance of designing and optimizing these more complex, hybrid frameworks. This is true especially today, with the ever rising cost of graphics hardware making high-fidelity graphics less accessible to the average consumer, these systems have become even more central for pushing this field forward.

Appendix A

Implementation Details

To complement the theoretical framework presented from Chapter 3 through Chapter 5, this appendix outlines the software architecture and design of the prototype rendering engine we developed, named *Felina*[10].

This appendix is structured in three main parts. Starting with Section A.1 we provide an overview of the entire project, presenting the technologies used throughout its development. Subsequently, in Section A.2 we detail the high-level design of the deferred rendering pipeline [49] and the last adjustments added to prepare it for integrating the global illumination framework. Lastly, in Section A.3 we thus briefly discuss how the presented theoretical framework may be integrated within *Felina*.

A.1 Overview

This prototype was written from scratch using modern C++20 and the Vulkan API [23]. Vulkan was chosen primarily for its cross-platform support, in contrast to its major competitors like DirectX (Microsoft) and Metal (Apple) being constrained to their specific operating systems (Windows and macOS/iOS, respectively).
On top of Vulkan, the renderer also incorporates the following libraries:

GLM (OpenGL Mathematics) Provides C++ data types and functions for linear algebra such as vector, matrix and also quaternions operations.

GLFW Handles cross-platform window creation by interfacing with the host operating system, providing the Vulkan window surface[11] and processing user input events.

[10] The code is hosted publicly at `https://github.com/Frol3z/Felina`.
[11] In Vulkan, rendering to a display requires an abstraction layer provided by platform-specific surface extensions (`VkSurfaceKHR`), which GLFW manages automatically [23].

Vulkan Memory Allocator (VMA) Simplifies explicit GPU memory management by providing high-level abstractions to handle memory allocations.

Dear ImGui Provides a lightweight graphical user interface to allow real-time interactions with the scene.

tinygltf Parses and loads 3D scenes and models, material properties and node hierarchies from glTF[12] asset files.

A.2 Architecture and Design

The architecture of *Felina* is organized around a modular design, delegating each responsibility across distinct components that interact with each other while keeping coupling to a minimum. A high-level description of the project is shown in Figure 13.

The architecture is logically split into three main components: the application, resource management and the renderer.

The application encompasses high-level tasks such as providing start-up and termination logic for the program, managing the main execution loop, coordinating input processing with other subsystems, handling errors and so on. Additionally, it acts as the interface between the rendering back-end and the front-end (e.g., user-interface, scene loading and window-level events). This layer was deliberately designed as a minimal, generic application to keep the primary development focus on the renderer. By keeping the two separated, we ensure that the renderer remains an independent module that can be, eventually, easily embedded into a user's custom application.

Resource management is almost entirely handled by two high-level abstractions: the `ResourceManager` and the `Scene`. The `Scene` is a description of the object hierarchy comprising the scene being rendered, including camera, objects and lights, which is usually populated dynamically when loading a glTF asset file. Because each object in the scene hierarchy consists of one or more meshes paired with a transformation matrix, and each mesh references a specific material, managing these shared assets required a centralized owner. The `ResourceManager` thus acts as the sole owner of different resource types (such as textures, meshes and materials), providing methods to allocate, deallocate and retrieve specific resources at runtime.

[12] glTF (`.gltf` / `.glb`) is an open-standard file format maintained by the Khronos Group for efficient transmission and loading of 3D scenes [22].

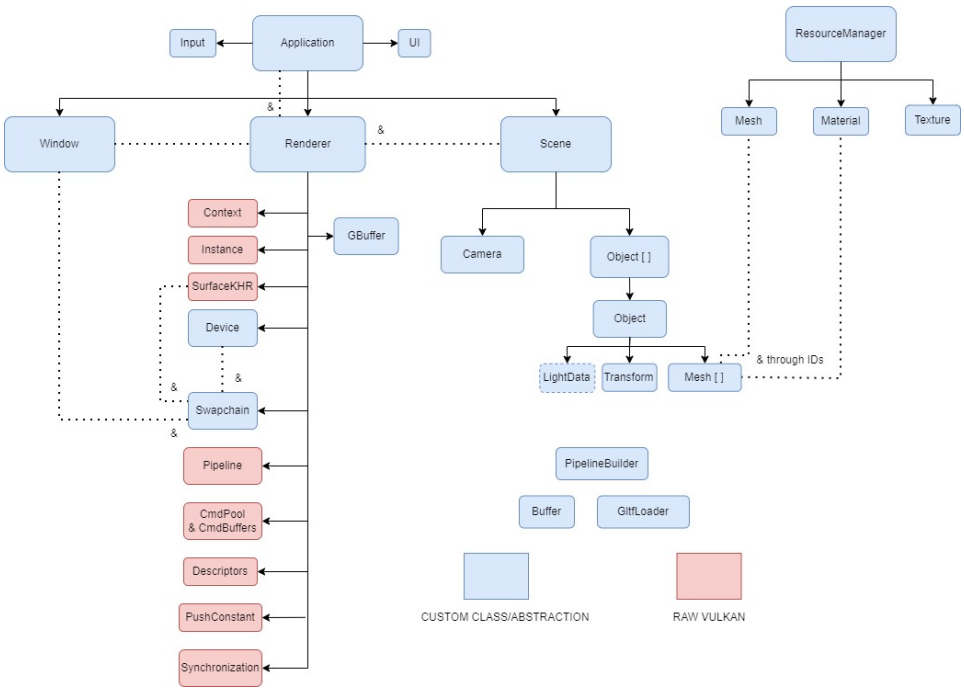


Figure 13: High-level diagram of the overall architecture. Here, the blue boxes encode our own abstractions, in contrast to the red boxes which represent low-level, raw Vulkan components. Additionally, solid arrows indicate direct ownership and composition (i.e., a child component is owned by its parent object), while dotted lines represent non-owning references, with the ampersand symbol (&) indicating which object holds the reference.

Lastly, the renderer encapsulates both high-level abstractions and low-level Vulkan objects to form a cohesive system with the goal of, given a scene description as input, rendering an image from the point of view of its camera and presenting it to the screen.

It does so by implementing a standard two-pass deferred rendering pipeline. The first pass consists of the rasterization of the scene into a set of textures known as render targets. This set of render targets is what constitutes the G-buffer and includes the albedo, material properties, normal and depth map. The second pass is a full-screen shading pass. For each pixel, it reads surface attributes directly from the G-buffer textures to evaluate the lighting model by considering all analytical light sources (i.e., directional, point and spot lights) within the scene. Each pass is implemented through a custom vertex and fragment shader written with Microsoft's high-level shader language (HLSL).

To prepare the renderer for integrating the global illumination framework, the Vulkan ray tracing back-end requires encapsulating the scene within a spatial acceleration structure hierarchy used to accelerate ray/triangle intersections.

Vulkan acceleration structures are grouped in two constructs [23]. The bottom-level acceleration structures (BLAS) are built for each unique mesh, directly from its vertex and index buffer data. On the other hand, the top-level acceleration structure (TLAS) is constructed dynamically to encapsulate world-space instances, each referencing a BLAS alongside its corresponding model transformation matrix. Consequently, while BLASes typically remain static throughout runtime, the TLAS must be rebuilt or updated every frame to account for dynamic objects. See Figure 14 for an example.

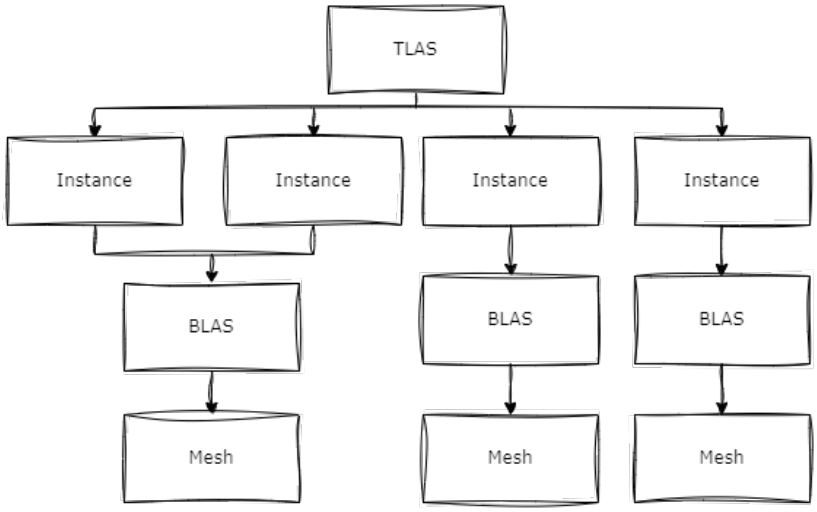


Figure 14: Illustration of a two-level spatial acceleration structure. Notice how a single BLAS may be referenced by multiple instances, for example, when representing the same mesh being instanced many times within the scene.

A.3 Framework Integration

To bridge the gap between *Felina* and the global illumination framework discussed within this work, this section now presents a blueprint detailing how the proposed probe-based solution may be integrated within our renderer.

We first begin by discussing the probe volume data structure. The cascaded probe volume must be shared between both CPU (host) and GPU (device) memory. While grid-specific parameters such as probe spacing, grid size and position may be stored within host memory and made accessible within shaders (e.g., through the usage of Vulkan descriptors [23]), the four overall atlases should be instead allocated directly on the device memory with the help of VMA. This is to ensure a faster memory access as all probe operations are performed exclusively on the GPU.

Since both probe selection and the subsequent update are completely independent from the screen-pass rasterization executed by the first pass of our rendering pipeline, they can be executed concurrently through the usage of Vulkan compute shaders. Ray tracing can be directly implemented within the compute kernels by using inline ray queries from the Vulkan device extension `VK_KHR_ray_query` [23]. Notice that both procedures require synchronization (e.g., implemented through the use of a barrier) before the

second pass of the pipeline begins. The next stage, in fact, strictly depends on both the updated probe volume caches and the G-buffer.

To extend the current local shading model to include global illumination as detailed in Chapter 5, this second pass must be expanded into multiple stages.

In the first stage, for each pixel, the indirect diffuse and specular ray are importance-sampled from their corresponding probability distributions and then cast into the scene. In contrast to the inline ray queries used during probe updates, this pass may be executed via a full Vulkan ray tracing pipeline (through the `VK_KHR_ray_tracing_pipeline` extension [23]). Utilizing dedicated closest-hit and miss shaders, the incoming indirect radiance is then sampled from the probe volume and, in case of indirect diffuse, also projected into SH-space [39]. The output of this first stage thus consists of three different render targets: one containing direct illumination, one for indirect specular radiance and one for indirect diffuse radiance encoded in SH.

In the next, second stage, these last two indirect lighting textures are then fed into a denoiser to produce a more stable indirect illumination.

Lastly, once the denoising process terminates, the final stage combines the denoised indirect lighting with the direct lighting to compute the final image.

As highlighted in Chapter 6, additional passes implementing screen-space techniques such as GTAO or SSR can be included into the rendering pipeline to further augment local details. Additionally, incorporating a dedicated shadowing algorithm, such as cascaded shadow mapping [53], should also be taken into consideration to increase the visual fidelity of the final image.

List of Acronyms

AABB	Axis-Aligned Bounding Box
BLAS	Bottom-Level Acceleration Structure
BRDF	Bidirectional Reflectance Distribution Function
BVH	Bounding Volume Hierarchy
DDGI	Dynamic Diffuse Global Illumination
DLSS-RR	Deep Learning Super Sampling - Ray Reconstruction
G-buffer	Geometric Buffer
GGX	Trowbridge–Reitz Distribution
GIBS	Global Illumination Based on Surfels
GPU	Graphics Processing Unit
GTAO	Ground-Truth Ambient Occlusion
HDR	High Dynamic Range
HLSL	High-Level Shader Language
IBL	Image-Based Lighting
LDS	Low-Discrepancy Sequences
LOD	Level of Detail
MC	Monte Carlo
NDF	Normal Distribution Function
NRD	NVIDIA Real-Time Denoisers
PDF	Probability Density Function
QMC	Quasi-Monte Carlo
SF	Spherical Fibonacci
SH	Spherical Harmonics
SPP	Samples Per Pixel
SSAO	Screen-Space Ambient Occlusion
SSR	Screen-Space Reflections
TLAS	Top-Level Acceleration Structure
VMA	Vulkan Memory Allocator

Bibliography

[1] Edward H. Adelson and James R. Bergen. "The Plenoptic Function and the Elements of Early Vision". In: *Computational Models of Visual Processing*. Ed. by Michael Landy and J. Anthony Movshon. The MIT Press, Oct. 1991. ISBN: 9780262290890. DOI: 10.7551/mitpress/2002.003.0004. eprint: https://direct.mit.edu/book/chapter-pdf/2318278/9780262290890_caa.pdf. URL: https://doi.org/10.7551/mitpress/2002.003.0004.

[2] Tomas Akenine-Möller et al. *Real-Time Rendering*. 4th. A K Peters/CRC Press, 2018. ISBN: 978-1138627000.

[3] Diede Apers. *Shipping Dynamic Global Illumination in Frostbite*. Presentation in ACM SIGGRAPH 2024 Course: Advances in Real-Time Rendering in Games. Aug. 2024. URL: https://advances.realtimerendering.com/s2024/content/EA-GIBS2/Apers_Advances-s2024_Shipping-Dynamic-GI.pdf.

[4] James F. Blinn. "Simulation of wrinkled surfaces". In: *SIGGRAPH Comput. Graph.* 12.3 (Aug. 1978), pp. 286–292. ISSN: 0097-8930. DOI: 10.1145/965139.507101. URL: https://doi.org/10.1145/965139.507101.

[5] James F. Blinn and Martin E. Newell. "Texture and reflection in computer generated images". In: *Commun. ACM* 19.10 (Oct. 1976), pp. 542–547. ISSN: 0001-0782. DOI: 10.1145/360349.360353. URL: https://doi.org/10.1145/360349.360353.

[6] Andreas Brinck et al. *Global Illumination Based on Surfels*. Presentation in ACM SIGGRAPH 2021 Course: Advances in Real-Time Rendering in Games. Aug. 2021. URL: https://advances.realtimerendering.com/s2021/SIGGRAPH%20Advances%202021%20-%20Surfel%20GI.pdf.

[7] Brent Burley. *Physically-Based Shading at Disney*. Course Notes. Walt Disney Animation Studios, 2012. URL: https://disneyanimation.com/publications/physically-based-shading-at-disney/.

[8] Zina H. Cigolle et al. "A Survey of Efficient Representations for Independent Unit Vectors". In: *Journal of Computer Graphics Techniques*

(JCGT) 3.2 (Apr. 2014), pp. 1–30. ISSN: 2331-7418. URL: http://jcgt.org/published/0003/02/01/.

[9] R. L. Cook and K. E. Torrance. "A Reflectance Model for Computer Graphics". In: *ACM Trans. Graph.* 1.1 (Jan. 1982), pp. 7–24. ISSN: 0730-0301. DOI: 10.1145/357290.357293. URL: https://doi.org/10.1145/357290.357293.

[10] Robert Cupisz. *Light Probe Interpolation Using Tetrahedral Tessellations*. Game Developers Conference (GDC). Unity Technologies, 2012. URL: https://www.gdcvault.com/play/1015312/Light-Probe-Interpolation-Using-Tetrahedral.

[11] Paul Debevec. "Rendering synthetic objects into real scenes: bridging traditional and image-based graphics with global illumination and high dynamic range photography". In: *Proceedings of the 25th Annual Conference on Computer Graphics and Interactive Techniques*. SIGGRAPH '98. New York, NY, USA: Association for Computing Machinery, 1998, pp. 189–198. ISBN: 0897919998. DOI: 10.1145/280814.280864. URL: https://doi.org/10.1145/280814.280864.

[12] William Donnelly and Andrew Lauritzen. "Variance shadow maps". In: *Proceedings of the 2006 Symposium on Interactive 3D Graphics and Games*. I3D '06. Redwood City, California: Association for Computing Machinery, 2006, pp. 161–165. ISBN: 159593295X. DOI: 10.1145/1111411.1111440. URL: https://doi.org/10.1145/1111411.1111440.

[13] Philip Dutre, Philippe Bekaert, and Kavita Bala. *Advanced Global Illumination*. 2nd. Natick, MA, USA: A K Peters/CRC Press, 2006. ISBN: 9781568813073. DOI: 10.1201/9781315365473.

[14] Thomas Engelhardt and Carsten Dachsbacher. "Octahedron Environment Maps". In: *Proceedings of the 13th International Workshop on Vision, Modeling, and Visualization (VMV)*. Konstanz, Germany: AKA Verlag, 2008, pp. 383–388.

[15] A. Gershun. "The Light Field". In: *Journal of Mathematics and Physics* 18.1-4 (1939), pp. 51–151. DOI: https://doi.org/10.1002/sapm193918151. eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1002/sapm193918151. URL: https://onlinelibrary.wiley.com/doi/abs/10.1002/sapm193918151.

[16] G. Greger et al. "The irradiance volume". In: *IEEE Computer Graphics and Applications* 18.2 (1998), pp. 32–43. DOI: 10.1109/38.656788.

[17] Eric Heitz. "Sampling the GGX Distribution of Visible Normals". In: *Journal of Computer Graphics Techniques (JCGT)* 7.4 (Nov. 2018), pp. 1–13. ISSN: 2331-7418. URL: http://jcgt.org/published/0007/04/01/.

[18] Michał Iwanicki and Angelo Pesce. "Approximate Models for Physically Based Rendering". In: *ACM SIGGRAPH 2015 Courses: Physically Based Shading in Theory and Practice*. New York, NY, USA: ACM, 2015. URL: https://c0de517e.blogspot.com/2016/07/siggraph-2015-notes-for-approximate.html.

[19] Jorge Jimenez et al. *Practical Realt-Tme Strategies for Accurate Indirect Occlusion*. Tech. rep. ATVI-TR-19-01. Activision Blizzard, 2019. URL: https://www.activision.com/cdn/research/Practical_Real_Time_Strategies_for_Accurate_Indirect_Occlusion_NEW%20VERSION_COLOR.pdf.

[20] James T. Kajiya. "The rendering equation". In: *Proceedings of the 13th Annual Conference on Computer Graphics and Interactive Techniques*. SIGGRAPH '86. New York, NY, USA: Association for Computing Machinery, 1986, pp. 143–150. ISBN: 0897911962. DOI: 10.1145/15922.15902. URL: https://doi.org/10.1145/15922.15902.

[21] Brian Karis. "Real Shading in Unreal Engine 4". In: *ACM SIGGRAPH 2013 Courses: Physically Based Shading in Theory and Practice*. New York, NY, USA: ACM, 2013. URL: https://cdn2.unrealengine.com/Resources/files/2013SiggraphPresentationsNotes-26915738.pdf.

[22] Khronos Group. *glTF 2.0 Specification*. The Khronos Group Inc. 2021. URL: https://registry.khronos.org/glTF/specs/2.0/glTF-2.0.html.

[23] Khronos Group. *Vulkan Specification*. The Khronos Group Inc. 2026. URL: https://registry.khronos.org/vulkan/specs/latest/html/vkspec.html.

[24] Ares Lagae and Philip Dutré. "A Comparison of Methods for Generating Poisson Disk Distributions". In: *Computer Graphics Forum* 27.1 (2008), pp. 114–129. DOI: https://doi.org/10.1111/j.1467-8659.2007.01100.x. eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1111/j.1467-8659.2007.01100.x. URL: https://onlinelibrary.wiley.com/doi/abs/10.1111/j.1467-8659.2007.01100.x.

[25] Sébastien Lagarde and Charles de Rousiers. *Moving Frostbite to PBR*. Course Notes. Electronic Arts Frostbite, 2014. URL: https://media.contentapi.ea.com/content/dam/eacom/frostbite/files/course-notes-moving-frostbite-to-pbr-v2.pdf.

[26] Dimitar Lazarov. "Getting More Physical in Call of Duty: Black Ops II". In: *ACM SIGGRAPH 2013 Courses: Physically Based Shading in Theory and Practice*. New York, NY, USA: ACM, 2013. DOI: 10.1145/2504435.2504439. URL: https://blog.selfshadow.com/publications/s2013-shading-course/lazarov/s2013_pbs_black_ops_2_notes.pdf.

[27] Luc LeBlanc and Melino Conte. *Ray Tracing the World of Assassin's Creed Shadows*. Presentation in ACM SIGGRAPH 2025 Course: Advances in Real-Time Rendering in Games. Aug. 2025. URL: https://advances.realtimerendering.com/s2025/content/Advances%202025%20-%20Raytracing%20the%20world%20of%20Assassin's%20Creed%20Shadows.pdf.

[28] Marc Levoy and Pat Hanrahan. "Light field rendering". In: *Proceedings of the 23rd Annual Conference on Computer Graphics and Interactive Techniques*. SIGGRAPH '96. New York, NY, USA: Association for Computing Machinery, 1996, pp. 31–42. ISBN: 0897917464. DOI: 10.1145/237170.237199. URL: https://doi.org/10.1145/237170.237199.

[29] Zander Majercik et al. "Dynamic Diffuse Global Illumination with Ray-Traced Irradiance Fields". In: *Journal of Computer Graphics Techniques (JCGT)* 8.2 (June 2019), pp. 1–30. ISSN: 2331-7418. URL: http://jcgt.org/published/0008/02/01/.

[30] Henrique S. Malvar and G J Sullivan. "Transform, Scaling & Color Space Impact of Professional Extensions". In: 2003. URL: https://api.semanticscholar.org/CorpusID:56572567.

[31] R. Marques et al. "Spherical Fibonacci Point Sets for Illumination Integrals". In: *Computer Graphics Forum* 32.8 (2013), pp. 134–143. DOI: https://doi.org/10.1111/cgf.12190. eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1111/cgf.12190. URL: https://onlinelibrary.wiley.com/doi/abs/10.1111/cgf.12190.

[32] Martin Mittring. *Finding Next Gen - CryEngine 2*. Presentation in ACM SIGGRAPH 2007 Course: Advances in Real-Time Rendering in Games. Aug. 2007. URL: https://advances.realtimerendering.

com/s2007/Mittring-Finding_NextGen_CryEngine2(Siggraph07%20Course%20Notes).pdf.

[33] NVIDIA. *NVIDIA Turing GPU Architecture: Technical Whitepaper*. Tech. rep. Accessed: 2026-09-24. NVIDIA Corporation, 2018. URL: https://images.nvidia.com/aem-dam/en-zz/Solutions/design-visualization/technologies/turing-architecture/NVIDIA-Turing-Architecture-Whitepaper.pdf.

[34] NVIDIA. *NVIDIA® Real-Time Denoisers (NRD)*. 2024. URL: https://github.com/NVIDIA-RTX/NRD (visited on 03/18/2026).

[35] NVIDIA. *NVIDIA® RTXGI*. 2024. URL: https://github.com/NVIDIAGameWorks/RTXGI (visited on 03/18/2024).

[36] Michael Oren and Shree K. Nayar. "Generalization of Lambert's reflectance model". In: *Proceedings of the 21st Annual Conference on Computer Graphics and Interactive Techniques*. SIGGRAPH '94. New York, NY, USA: Association for Computing Machinery, 1994, pp. 239–246. ISBN: 0897916670. DOI: 10.1145/192161.192213. URL: https://doi.org/10.1145/192161.192213.

[37] Art B. Owen. *Monte Carlo theory, methods and examples*. Self-published, 2013. URL: https://artowen.su.domains/mc/.

[38] Matt Pharr, Wenzel Jakob, and Greg Humphreys. *Physically Based Rendering: From Theory to Implementation*. 4th. Cambridge, MA, USA: MIT Press, 2023. ISBN: 9780262048026. URL: https://mitpress.mit.edu/9780262048026/physically-based-rendering/.

[39] Ravi Ramamoorthi and Pat Hanrahan. "An efficient representation for irradiance environment maps". In: *Proceedings of the 28th Annual Conference on Computer Graphics and Interactive Techniques*. SIGGRAPH '01. New York, NY, USA: Association for Computing Machinery, 2001, pp. 497–500. ISBN: 158113374X. DOI: 10.1145/383259.383317. URL: https://doi.org/10.1145/383259.383317.

[40] Takafumi Saito and Tokiichiro Takahashi. "Comprehensible rendering of 3-D shapes". In: *Proceedings of the 17th Annual Conference on Computer Graphics and Interactive Techniques*. SIGGRAPH '90. Dallas, TX, USA: Association for Computing Machinery, 1990, pp. 197–206. ISBN: 0897913442. DOI: 10.1145/97879.97901. URL: https://doi.org/10.1145/97879.97901.

[41] Christoph Schied et al. "Spatiotemporal Variance-Guided Filtering: Real-time Reconstruction for Path Traced Global Illumination". In: *ACM/EG Symposium on High Performance Graphics (HPG)*. July 2017, pp. 23–41. DOI: 10.1145/3105762.3105770.

[42] Christophe Schlick. "An Inexpensive BRDF Model for Physically-based Rendering". In: *Computer Graphics Forum* 13.3 (1994), pp. 233–246. DOI: https://doi.org/10.1111/1467-8659.1330233. eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1111/1467-8659.1330233. URL: https://onlinelibrary.wiley.com/doi/abs/10.1111/1467-8659.1330233.

[43] Perumaal Shanmugam and Okan Arikan. "Hardware accelerated ambient occlusion techniques on GPUs". In: *Proceedings of the 2007 Symposium on Interactive 3D Graphics and Games*. I3D '07. Seattle, Washington: Association for Computing Machinery, 2007, pp. 73–80. ISBN: 9781595936288. DOI: 10.1145/1230100.1230113. URL: https://doi.org/10.1145/1230100.1230113.

[44] Peter-Pike Sloan. *Stupid Spherical Harmonics (SH) Tricks*. Game Developers Conference (GDC). San Francisco, CA, USA, Feb. 2008. URL: https://www.ppsloan.org/publications/StupidSH36.pdf.

[45] Peter-Pike Sloan, Derek Nowrouzezahrai, and Hong Yuan. "Wrap Shading". In: *Journal of Graphics, GPU, and Game Tools* 15.4 (2011), pp. 252–259. DOI: 10.1080/2151237X.2011.628841. URL: https://doi.org/10.1080/2151237X.2011.628841.

[46] I.M Sobol'. "On the distribution of points in a cube and the approximate evaluation of integrals". In: *USSR Computational Mathematics and Mathematical Physics* 7.4 (1967), pp. 86–112. ISSN: 0041-5553. DOI: https://doi.org/10.1016/0041-5553(67)90144-9. URL: https://www.sciencedirect.com/science/article/pii/0041555367901449.

[47] Tiago Sousa. *FAST AS HELL: idTech 8 Global Illumination*. Presentation in ACM SIGGRAPH 2025 Course: Advances in Real-Time Rendering in Games. Aug. 2025. URL: https://advances.realtimerendering.com/s2025/content/SOUSA_SIGGRAPH_2025_Final.pdf.

[48] Tiago Sousa, Nickolay Kasyan, and Nicolas Schulz. "Secrets of CryENGINE 3 Graphics Technology". In: *SIGGRAPH Advances in Real-Time Rendering in 3D Graphics and Games course*. 2011.

[49] Jonathan Thaler. "Deferred Rendering". Master's thesis. Linz, Austria: Johannes Kepler University Linz, Feb. 2011. URL: https://www.researchgate.net/publication/323357208_Deferred_Rendering.

[50] Bruce Walter et al. "Microfacet models for refraction through rough surfaces". In: *Proceedings of the 18th Eurographics Conference on Rendering Techniques*. EGSR'07. Grenoble, France: Eurographics Association, 2007, pp. 195–206. ISBN: 9783905673524.

[51] Lance Williams. "Casting curved shadows on curved surfaces". In: SIGGRAPH '78. New York, NY, USA: Association for Computing Machinery, 1978, pp. 270–274. ISBN: 9781450379083. DOI: 10.1145/800248.807402. URL: https://doi.org/10.1145/800248.807402.

[52] Daniel Wright, Krzysztof Narkowicz, and Patrick Kelly. *Real-time Global Illumination in Unreal Engine 5*. Presentation in ACM SIGGRAPH 2022 Course: Advances in Real-Time Rendering in Games. Aug. 2022. URL: https://advances.realtimerendering.com/s2022/SIGGRAPH2022-Advances-Lumen-Wright%20et%20al.pdf.

[53] Fan Zhang et al. "Parallel-split shadow maps for large-scale virtual environments". In: *Proceedings of the 2006 ACM International Conference on Virtual Reality Continuum and Its Applications*. VRCIA '06. Hong Kong, China: Association for Computing Machinery, 2006, pp. 311–318. ISBN: 1595933247. DOI: 10.1145/1128923.1128975. URL: https://doi.org/10.1145/1128923.1128975.